Directed Graph Hashing

Caleb Helbling*

Abstract. We present methods, with correctness proofs, for hashing node-labeled directed graphs and the individual nodes within them. We distinguish between hashing based on the *global* structure of a graph, where equal hashes correspond to isomorphism and automorphism orbits, and hashing based on *local* observations obtained by traversing the graph along its edges, where equal hashes correspond to isomorphic (possibly infinite) unfoldings or, in a variant, to bisimilarity. We give Merkle-style hashes that recurse over the condensation of a graph and hash each strongly connected component with a component hash, and we characterize exactly which nodes receive equal hashes, in a global and a local variant. Applying the local variant, with a component hash based on canonical colour refinement, to the quotient of the graph by unfolding isomorphism, we obtain local hashes of all nodes in $O((n+m)\log n)$ time for a graph with n nodes and m edges. For graphs whose outgoing edges are ordered we give linear-time node hashes and a canonization of strongly connected graphs in $O((n+m)\log n + c\,(n+m))$ time, where c is the size of a smallest class of nodes with isomorphic ordered unfoldings, and we show that canonizing general ordered digraphs is as hard as canonizing graphs. We also show that hashing nodes by the language of their walks is PSPACE-hard. Structural hashing of trees is ubiquitous in blockchains, version control, and content-addressed storage; by contrast, hashing of cyclic directed graphs has received comparatively little systematic treatment. An implementation is validated against brute-force equivalence checks, and we report its running time on random digraphs and on worst-case families.

1 Introduction Hashing functions are fundamental building blocks in the modern software stack that have seen widespread use in data storage and retrieval, digital fingerprinting, checksum and cryptographic applications. Traditionally, hash functions take as input bit strings of arbitrary length and return a bit string of a fixed size $\mathcal{H} : \{0,1\}^* \to \{0,1\}^n$. Good hashing functions should be fast to compute, minimize the chance of duplication of output values, and produce large changes in output for small changes in input. Many bit string hashing functions have been created over the years, including well known algorithms such as MD5 [37], SHA-2 [33], and SHA-3 [34].

Algorithm 1.1 Recursive Merkle-tree Hashing of Ordered Binary Tree

$$\mathrm{mh}(n) = \begin{cases} \mathcal{H}(n.\mathrm{label}) & n \text{ is a leaf} \\ \mathcal{H}(n.\mathrm{label} \circ \mathrm{mh}(n.\mathrm{left}) \circ \mathrm{mh}(n.\mathrm{right})) & \text{otherwise} \end{cases}$$

Hashing functions can be extended to trees via a simple extension of Merkle hashing [30]. In Merkle tree hashing, the hash of a node is computed by hashing the concatenation of the node's label with the recursive hash of the node's children. Algorithm 1.1 gives a simple recursive implementation of Merkle tree hashing for ordered binary trees. This particular variation of Merkle hashing has seen use in many applications, including cryptocurrency [31], version control systems [25], and distributed hypertext document storage systems [1]. For *unordered* trees, replacing the sequence of child hashes by a sorted multiset of child hashes yields a hash that is invariant under tree isomorphism, in the spirit of the classical Aho–Hopcroft–Ullman tree isomorphism algorithm [3].

Extending hashing to node-labeled directed graphs is non-trivial due to the presence of cycles: the recursion of Algorithm 1.1 no longer terminates. Indeed, the design space for hashing digraphs is surprisingly broad and depends on use case objectives. In this paper we focus on the following approaches, which we think are the most useful for practical applications:

*Draper (chelbling@draper.com).

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

- **Global Hashing** (Section 3) — for use cases where the *global* structure of the digraph is of importance. Two graphs hash equally if and only if they are isomorphic, and two nodes hash equally if and only if some isomorphism of their graphs maps one onto the other; in particular, nodes in the same automorphism orbit hash equally.

- **Merkle Graph Hashing** (Section 4) — for use cases where graphs are hashed recursively, or are built up over time. We hash strongly connected components individually with a component hash, then recurse over the acyclic condensation graph, and characterize precisely which nodes receive equal hashes for any component hash. With the global node hash of each component this gives an equivalence that lies strictly between rooted isomorphism and unfolding isomorphism.

- **Local Hashing** (Section 5) — for use cases where observers can only observe *local* information by traversing from a node to a neighbor on an outgoing edge. Two nodes hash equally if and only if the (potentially infinite) trees obtained by unfolding the graph from them are isomorphic. We obtain this hash from the Merkle construction, with a canonical local component hash, applied to the quotient of the graph by unfolding isomorphism. We also give a bisimilarity variant, and show that both can be computed in $O((n+m)\log n)$ time for a graph with n nodes and m edges. Additionally, we show that hashing the language of vertex walks is PSPACE-hard. Local hashing has many commonalities with bisimulation and co-inductive representations of finite directed graphs [38, 36].

- **Ordered Outgoing Edges** (Section 6) — in many practical applications the outgoing edges from a node are ordered. We use this additional structure to obtain linear-time node hashes. For strongly connected graphs, we give an algorithm for computing both a canonical order and orbit parition in near-linear time for most graphs, and quadratic time in the worst case. This algorithm can then be plugged into our Merkle hashing approach to obtain an efficient global Merkle hash. For general ordered digraphs, canonization is as hard as for graphs.

One common theme throughout this paper is reliance on conversion of graphs to *canonical form*. If two graphs (or nodes within them) can be converted to a canonical form, we can serialize the canonical form and pass it through an ordinary string hashing function. We distinguish carefully between the hash of an entire graph and the hash of a specific node within a graph. Our general strategy for computing the hash of a specific node is to combine the overall graph hash with the node's position in canonical order, taking care that this position accounts for any symmetry present in the graph (two nodes may be identical modulo automorphism).

Contributions. Each hashing scheme comes with an exact characterization of hash equality (Theorems 3.1, 3.2, 5.15, 5.17, 6.2, 6.3, 6.4 and 4.2), stated relative to an idealized string hash (Section 2.4). We compute local hashes, which can be compared across graphs, for unfolding isomorphism and for bisimilarity in $O((n+m)\log n)$ time (Theorems 5.15 and 5.17), using the canonical colour refinement of Berkholz, Bonsma and Grohe [8] (Lemma 5.7) and the local Merkle hash of the quotient (Corollary 5.14); the Merkle theorem (Theorem 4.2) holds for any component hash. We prove PSPACE-hardness of hashing walk languages (Proposition 5.16), canonize strongly connected ordered graphs with searches from a single class of nodes (Theorem 6.4), show that canonizing general ordered digraphs is as hard as canonizing graphs (Proposition 6.5), characterize the local Merkle equivalence by SCC-annotated unfoldings (Proposition 5.11), and position both Merkle schemes relative to the other equivalences (Proposition 5.12 and Corollary 5.14). An implementation is validated against brute-force checks and benchmarked in Section 9.

2 Preliminaries

2.1 Labeled digraphs and symmetry. A *labeled digraph* is a triple $G = (V, E, L)$ where V is a finite vertex set, $E \subseteq V \times V$ is the edge set (self-loops are allowed), and L maps vertices to labels drawn from a set of strings. From Section 4 on, we also use *labeled multidigraphs* $G = (V, \mu, L)$, where $\mu : V \times V \to \mathbb{N}$ gives edge multiplicities; a digraph is a special case. We write $\mathrm{succ}(u)$ for the multiset of successors of u (with v occurring $\mu(u,v)$ times), and $R(u)$ for the set of vertices reachable from u (including u). A set $S \subseteq V$ is *forward-closed* if every successor of a vertex of S lies in S; each $R(u)$ is forward-closed, and so is V.

An *isomorphism* $\phi : G_1 \to G_2$ is a bijection $V_1 \to V_2$ with $L_1(u) = L_2(\phi(u))$ and $\mu_1(u,v) = \mu_2(\phi(u), \phi(v))$ for all u, v; we write $G_1 \cong G_2$. A *rooted isomorphism* $(G_1, v_1) \cong (G_2, v_2)$ is an isomorphism with $\phi(v_1) = v_2$. An

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

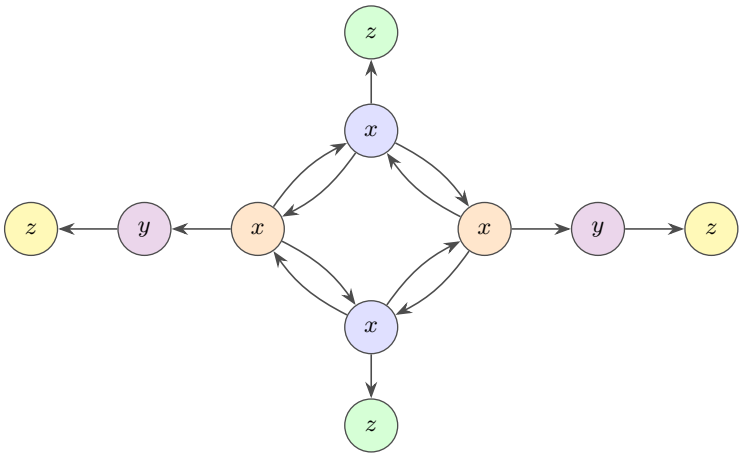


Figure 2.1: A directed graph with three node labels (x, y, z, shown as letters) and five automorphism orbits (shown by fill colour). Its automorphism group has order four and is generated by the two reflections of the drawing (their product is the rotation by 180°). Equal labels do not imply equal orbits: the four x-labelled nodes of the central diamond fall into two orbits, and the four z-labelled sinks fall into two orbits, because an x-node's pendant neighbour is a sink in one case and has a successor in the other.

automorphism is an isomorphism $G \to G$; the automorphisms form the group

$$\mathrm{Aut}(G) = \{\sigma : V \to V \text{ bijective} \mid L(\sigma(u)) = L(u),\ (u,v) \in E \leftrightarrow (\sigma(u), \sigma(v)) \in E\}.$$

Informally, the automorphism group gives all of the ways that G is symmetrical. The *orbit* of a vertex is the set of vertices it can be mapped to by automorphisms,

$$\mathrm{Orb}(G, v) = \{\sigma(v) \mid \sigma \in \mathrm{Aut}(G)\},$$

and the orbits partition V. Figure 2.1 gives an example. Note that v_1 and v_2 lie in the same orbit of G if and only if $(G, v_1) \cong (G, v_2)$.

2.2 Canonical forms. Fix a canonical labeling procedure (for example nauty/Traces [28, 29], bliss [20], or saucy [15], all of which support vertex labels). Given G with $|V| = n$ these algorithms return a bijection $\rho : V \to \{0, \dots, n-1\}$ such that the relabeled graph $\mathrm{canon}(G) = \rho(G)$ satisfies

$$\mathrm{canon}(G_1) = \mathrm{canon}(G_2) \iff G_1 \cong G_2.$$

By construction ρ is an isomorphism $G \to \mathrm{canon}(G)$. It is determined only up to automorphisms: if ρ is a canonical labeling then so is $\rho \circ \sigma$ for every $\sigma \in \mathrm{Aut}(G)$, and no canonical labeling procedure can prefer one of these over another. This is precisely why node hashes must be defined through orbits rather than through canonical indices alone.

2.3 Strongly connected components. A *strongly connected component* (SCC) of a digraph is a maximal set of vertices such that there is a path between any two of them. The *condensation graph* is formed by contracting each SCC into a single node; it is acyclic, and all SCCs together with the condensation can be computed in linear time [41]. The *height* of an SCC is the number of edges on a longest path starting from it in the condensation. Figure 2.2 shows an example.

2.4 The hashing model. All of our schemes serialize a finite structure and apply a string hash $\mathcal{H}$. We assume a serialization $\mathrm{enc}(\cdot)$ of nested tuples, lists, integers and strings that is *injective* (for example, prefixing every item with a type tag and its length, so that labels containing delimiter characters cannot cause ambiguity), and we encode a multiset as the sorted list of its elements. Since $\mathcal{H}$ maps into a finite set it cannot be injective, so we state our theorems in the following sense.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

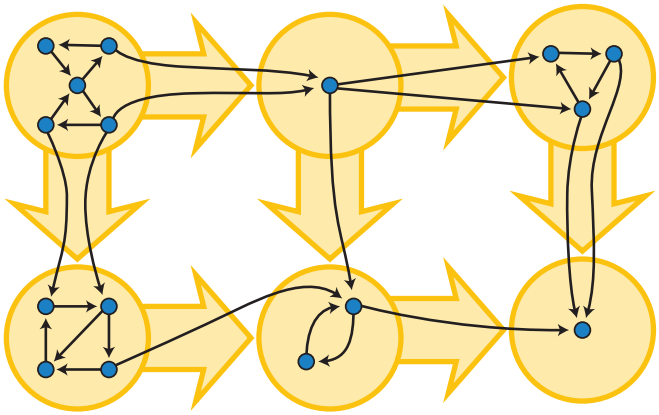


Figure 2.2: Each yellow node contains a strongly connected component of a graph. When each strongly connected component is contracted into a single node, the yellow condensation graph is formed. Since the condensation graph is acyclic, it can be recursed over. Public domain figure courtesy of David Eppstein.[2]

Definition 2.1 (Correctness up to collisions). A hash h *realizes* an equivalence relation $\sim$ if (i) $x \sim y$ implies $h(x) = h(y)$, and (ii) whenever $h(x) = h(y)$ but $x \not\sim y$, the computations of $h(x)$ and $h(y)$ contain two distinct strings $s \neq s'$ with $\mathcal{H}(s) = \mathcal{H}(s')$.

Condition (ii) says that a failure of the "only if" direction exhibits an explicit collision of $\mathcal{H}$, which is believed infeasible for SHA-2 or SHA-3. Equivalently, every statement "$h(x) = h(y)$ if and only if $x \sim y$" below is proved under the idealization that $\mathcal{H}$ is injective, and each proof of the "only if" direction uses injectivity only by comparing two strings whose hashes are equal. For readability we say "assuming $\mathcal{H}$ is injective" in the proofs.

3 Global Graph Hashing In global graph hashing, we would like our computed hash values to be unique modulo graph isomorphism: $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ if and only if $G_1 \cong G_2$. We canonize the input digraph and construct an adjacency list sorted in canonical order. The node labels (in canonical order) and the canonical adjacency list are then serialized and hashed using a string hashing function $\mathcal{H}$. Algorithm 3.1 gives an implementation of this approach. Note that the pair (labels, adj_list) is exactly an encoding of canon(G): the number of vertices is the length of the label list, so isolated vertices are accounted for.

THEOREM 3.1 (Correctness of the global graph hash). *$\mathcal{H}_G$ realizes graph isomorphism: assuming $\mathcal{H}$ is injective, $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ if and only if $G_1 \cong G_2$.*

Proof. ($\Rightarrow$) If $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$, then by injectivity of $\mathcal{H}$ and enc the canonical label lists and adjacency lists coincide, i.e. canon(G_1) = canon(G_2). Hence $G_1 \cong G_2$.

($\Leftarrow$) If $G_1 \cong G_2$ then canon(G_1) = canon(G_2), so the label lists and sorted adjacency lists coincide, and so do their encodings and hashes. ∎

Complexity. Since $\mathcal{H}_G$ decides isomorphism of labeled digraphs (in the idealized model), computing it is at least as hard as graph isomorphism (GI). No polynomial-time algorithm for GI is known; the best known worst-case bound is quasipolynomial, both for isomorphism testing [6] and for canonization [7]. Practical canonization

[2] https://commons.wikimedia.org/wiki/File:Graph_Condensation.svg

Algorithm 3.1 Global Digraph Hash

```
1: function H_G(G = (V, E, L))
2:     ρ ← CANONIZE(G)                          ▷ canonical labeling ρ : V → {0, ..., |V| − 1}
3:     adj_list ← SORT([(ρ(u), ρ(v)) | (u, v) ∈ E])
4:     labels ← [L(ρ^{-1}(i)) for i = 0, ..., |V| − 1]
5:     return H(enc((labels, adj_list)))
6: end function
```

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 3.2 Global Digraph Node Hash

```
1: function H_N(G, v)
2:     ρ ← CANONIZE(G)                          ▷ the same ρ is used in the next line
3:     g_hash ← H_G(G)                          ▷ computed from ρ as in Algorithm 3.1
4:     orbit_idx ← min Orb(canon(G), ρ(v))
5:     return H(enc((orbit_idx, g_hash)))
6: end function
```

tools based on individualization and refinement [28, 29, 20, 15] are extremely fast on most inputs, but have exponential worst-case running time on specially constructed families. The same remarks apply to computing orbits, which Algorithm 3.2 below requires; tools such as nauty return the orbit partition as a by-product of canonization.

The algorithm for hashing nodes builds on the algorithm for hashing graphs. First, the graph is canonized and the hash of the canonical graph is computed using Algorithm 3.1. Then we compute the *orbit index* of the input vertex: the smallest canonical index in the orbit of $\rho(v)$ in the canonical graph $\mathrm{canon}(G)$. (Equivalently, it is $\min\{\rho(u) \mid u \in \mathrm{Orb}(G, v)\}$, since ρ maps orbits of G onto orbits of $\mathrm{canon}(G)$.) The orbit index and the graph hash are then combined and passed through a string hashing function. Algorithm 3.2 gives an implementation of this approach. The orbit index does not depend on which canonical labeling $\rho \circ \sigma$, $\sigma \in \mathrm{Aut}(G)$, the canonizer happens to return, since $\rho(\sigma(v))$ lies in the same orbit of $\mathrm{canon}(G)$ as $\rho(v)$.

THEOREM 3.2 (Correctness of the global node hash). *$\mathcal{H}_N$ realizes rooted isomorphism: assuming $\mathcal{H}$ is injective, $\mathcal{H}_N(G_1, v_1) = \mathcal{H}_N(G_2, v_2)$ if and only if there is an isomorphism $\phi : G_1 \to G_2$ with $\phi(v_1) = v_2$. In particular, two nodes of the same graph hash equally if and only if they lie in the same orbit.*

Proof. Let ρ_1, ρ_2 be the canonical labelings of G_1, G_2.

($\Leftarrow$) Since $G_1 \cong G_2$, $C := \mathrm{canon}(G_1) = \mathrm{canon}(G_2)$ and $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ by Theorem 3.1. The map $\alpha = \rho_2 \circ \phi \circ \rho_1^{-1}$ is a composition of isomorphisms $C \to G_1 \to G_2 \to C$, hence $\alpha \in \mathrm{Aut}(C)$, and $\alpha(\rho_1(v_1)) = \rho_2(v_2)$. Thus $\rho_1(v_1)$ and $\rho_2(v_2)$ lie in the same orbit of C, so the orbit indices coincide, and so do the node hashes.

($\Rightarrow$) Equal node hashes imply, by injectivity, equal graph hashes and equal orbit indices. By Theorem 3.1 (and its proof) $\mathrm{canon}(G_1) = \mathrm{canon}(G_2) =: C$. Two orbits of C with the same minimum element are the same orbit, so there is $\alpha \in \mathrm{Aut}(C)$ with $\alpha(\rho_1(v_1)) = \rho_2(v_2)$. Then $\phi = \rho_2^{-1} \circ \alpha \circ \rho_1$ is an isomorphism $G_1 \to G_2$ with $\phi(v_1) = v_2$. ∎

4 Merkle Graph Hashing For applications where graphs are built up over time, a recursive approach similar to Merkle-tree hashing is highly beneficial. We now construct such an algorithm, which recurses over the condensation graph. The condensation is acyclic, so the recursion is well founded. The construction is parameterized by a *component hash*, which hashes the nodes of a single strongly connected graph. With the global node hash of Section 3 it gives the global Merkle hash of this section. With the local component hash of Section 5 it gives the local Merkle hash, and, applied to the unfolding quotient, the local hash itself.

To compute the hash of each node, we process the strongly connected components in reverse topological order, so that every edge leaving a component points to nodes that have already been hashed. Each node x of the component is relabeled by combining its label with the multiset $X(x)$ of the hashes of its external targets, counted with multiplicity. The new label $L'(x) = (L(x), X(x))$ is kept as a pair rather than hashed. The canonical form of a component therefore contains every label and every external hash verbatim, so the component, including its outgoing links, can be reconstructed from the value that is hashed (Section 7 uses this). The labels L' are structured values rather than strings. This is harmless, because the component hashes of Sections 3, 5 and 6 use labels only through equality, sorting and the injective serialization enc function. Finally the relabeled component, restricted to its internal edges, is hashed with the component hash. Algorithm 4.1 gives the pseudo-code; each component is processed once. Section 5 also uses a *set variant*, in which $X(x)$ is the set rather than the multiset of external hashes, and Section 6 adapts the algorithm to ordered graphs.

Which nodes receive equal hashes? Across components, a Merkle hash behaves like a Merkle tree: it compares external targets only through their hashes and ignores whether two targets are shared. Within a component, it inherits the equivalence realized by the component hash parameter. We make this precise for an arbitrary component hash. Fix labeled multidigraphs G_1 and G_2 (possibly equal), and let $W = V_1 \sqcup V_2$. For $u \in W$ let

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 4.1 Merkle Graph Node Hashing

```
1: function H_M(G = (V, μ, L), COMP_HASH)
2:     h ← EMPTY_DICT()
3:     for each SCC S in reverse topological order do          ▷ Tarjan's algorithm; sinks first
4:         for x ∈ S do
5:             X(x) ← {{h[y] | y ∉ S}}, where h[y] is repeated μ(x, y) times
6:             L'(x) ← (L(x), X(x))                            ▷ not hashed
7:         end for
8:         G' ← (S, μ|_{S×S}, L')
9:         h ← h ∪ COMP_HASH(G')                               ▷ hashes of all x ∈ S
10:    end for
11:    return h
12: end function
```

$\mathrm{scc}(u)$ be the SCC of u in its own graph, and let $\mathrm{ht}(u)$ be the height of $\mathrm{scc}(u)$.

Algorithm 4.1 passes the component hash the labels $L'(x)$, which contain hash values, whereas the equivalence we want to characterize is defined on the classes that these hashes stand for. We therefore constrain COMP_HASH so that hash equality is unaffected by an injective relabeling of the input strongly connected component: two nodes have equal hashes before the relabeling if and only if they have equal hashes after it.

Definition 4.1 (Component and Merkle equivalences). A *component equivalence* is an equivalence relation $\sim$ on pairs (S, x), where S is a strongly connected labeled multidigraph and x is a vertex of S, that is preserved and reflected by injective relabeling: if f is an injective map on labels and $f(S)$ denotes S with every label ℓ replaced by $f(\ell)$, then $(S, x) \sim (S', x')$ if and only if $(f(S), x) \sim (f(S'), x')$. A component hash *realizes* $\sim$ if, assuming $\mathcal{H}$ is injective, $\text{COMP_HASH}(S_1)[x_1] = \text{COMP_HASH}(S_2)[x_2]$ if and only if $(S_1, x_1) \sim (S_2, x_2)$.

The *Merkle equivalence* $\approx$ of $\sim$ is the relation on W defined by well-founded recursion on $\max(\mathrm{ht}(u), \mathrm{ht}(v))$. Suppose that $\approx$ has been defined on the nodes of height less than h. For a node x of height at most h, all external successors of x have height less than h. Let $X^{\approx}(x)$ be the multiset of their $\approx$-classes, counted with multiplicity, and let $C^{\approx}(x)$ be the sub-multigraph induced on $\mathrm{scc}(x)$, with labels $L^{\approx}(y) = (L(y), X^{\approx}(y))$. For u, v with $\max(\mathrm{ht}(u), \mathrm{ht}(v)) = h$, set

$$u \approx v \iff (C^{\approx}(u), u) \sim (C^{\approx}(v), v).$$

In the set variant, $X^{\approx}(x)$ is the set of these classes. We write $\approx_{\mathrm{glob}}$ for the Merkle equivalence of rooted isomorphism.

Rooted isomorphism, isomorphism of unfoldings and bisimilarity (Section 5) are component equivalences. The recursion is well founded because external successors lie in SCCs of strictly smaller height, and by induction on h, $\approx$ is an equivalence relation. For these three component equivalences, $\approx$ moreover relates only nodes of equal height. Indeed, if $u \approx v$ then every node of $\mathrm{scc}(u)$ corresponds to a node of $\mathrm{scc}(v)$ with the same label $L^{\approx}$ and vice versa (for unfoldings and bisimilarity, because in a strongly connected graph every node is reachable from every other). So the two components have the same classes of external successors, and hence the same height.

THEOREM 4.2 (Correctness of the Merkle hashes). *Let* COMP_HASH *realize a component equivalence* $\sim$*, and let* $\approx$ *be the Merkle equivalence of* $\sim$*. Assuming* $\mathcal{H}$ *is injective,* $\mathcal{H}_M(G_1, \text{COMP_HASH})[u] = \mathcal{H}_M(G_2, \text{COMP_HASH})[v]$ *if and only if* $u \approx v$*. The same holds for the set variant.*

Proof. By induction on $h = \max(\mathrm{ht}(u), \mathrm{ht}(v))$. By the induction hypothesis (vacuous when $h = 0$), two nodes of height less than h have equal hashes if and only if they are $\approx$-equivalent. So $[y]_{\approx} \mapsto h[y]$ is a well-defined injective map on these classes. Hence, for nodes x, x' of height at most h, $X(x) = X(x')$ if and only if $X^{\approx}(x) = X^{\approx}(x')$, and since $L'(x) = (L(x), X(x))$, $L'(x) = L'(x')$ if and only if $L^{\approx}(x) = L^{\approx}(x')$. Thus the relabeled components G' of Algorithm 4.1 that contain u and v are obtained from $C^{\approx}(u)$ and $C^{\approx}(v)$ by a single injective relabeling, which maps $L^{\approx}(x)$ to $L'(x)$. Since $\sim$ is preserved and reflected by injective relabeling, $(G'(u), u) \sim (G'(v), v)$ if and only if $(C^{\approx}(u), u) \sim (C^{\approx}(v), v)$, that is, if and only if $u \approx v$. Since the component hash realizes $\sim$, this holds if and only if $h[u] = h[v]$. The same argument applies to sets in place of multisets. ∎

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

The global Merkle hash. In the *global variant* $\mathcal{H}_M^{\text{glob}}$, the component hash is the global node hash (Algorithm 3.2), which realizes rooted isomorphism by Theorem 3.2. Theorem 4.2 therefore gives the following.

COROLLARY 4.3. *Assuming $\mathcal{H}$ is injective, $\mathcal{H}_M^{\text{glob}}$ realizes $\approx_{\text{glob}}$ on labeled digraphs.*

Spelled out, $u \approx_{\text{glob}} v$ if and only if there is a bijection $\phi : \text{scc}(u) \to \text{scc}(v)$ with $\phi(u) = v$ that is a label-preserving isomorphism of the induced subgraphs and, for every $x \in \text{scc}(u)$, matches the external successors of x bijectively with $\approx_{\text{glob}}$-equivalent external successors of $\phi(x)$. The global variant thus respects the full structure inside each component, including automorphisms and incoming edges inside the component. Proposition 5.12 in Section 5.5 places $\approx_{\text{glob}}$ between rooted isomorphism of reachable subgraphs and isomorphism of unfoldings.

Complexity and incrementality. Computing the SCCs takes $O(n+m)$ time, and each component is relabeled and hashed once. The total cost is $O((n+m)\log n)$ for the relabeling plus $\sum_S c(S)$, where $c(S)$ is the cost of the component hash on S, plus the cost of hashing. For the global variant, $c(S)$ is the cost of Algorithm 3.2 on S, which is GI-hard in general. The local variant of Section 5.5 runs in $O((n+m)\log n)$ time, and Section 6 gives the costs for ordered graphs. As with Merkle trees, changing a node changes only the hashes of nodes from which it is reachable. Adding a new node whose edges point only to existing nodes creates a new singleton SCC and leaves all existing hashes unchanged. Merkle hashes deliberately forget whether subgraphs below a component are shared, exactly as Merkle trees and Merkle DAGs do in git and IPFS.

5 Local Graph Hashing In comparison to the global hashing strategy which depends on the global structure of the graph, the local hashing strategy depends only on properties we can *observe* by traversing the digraph forward along its edges. To make this concept precise, the hashes of two nodes should be equal if and only if their *unfoldings* are isomorphic. We characterize this equivalence by stable partitions and merge equivalent nodes into a quotient (Section 5.2). Canonical colour refinement [8] then orders the nodes of a strongly connected quotient, which gives a component hash (Sections 5.3 and 5.4). With this component hash, the Merkle hash of Section 4 realizes an SCC-annotated variant of unfolding isomorphism (Section 5.5), and applied to the quotient it realizes unfolding isomorphism itself (Section 5.6). The resulting hash runs in $O((n+m)\log n)$ time for a graph with n nodes and m edges.

5.1 Unfoldings.

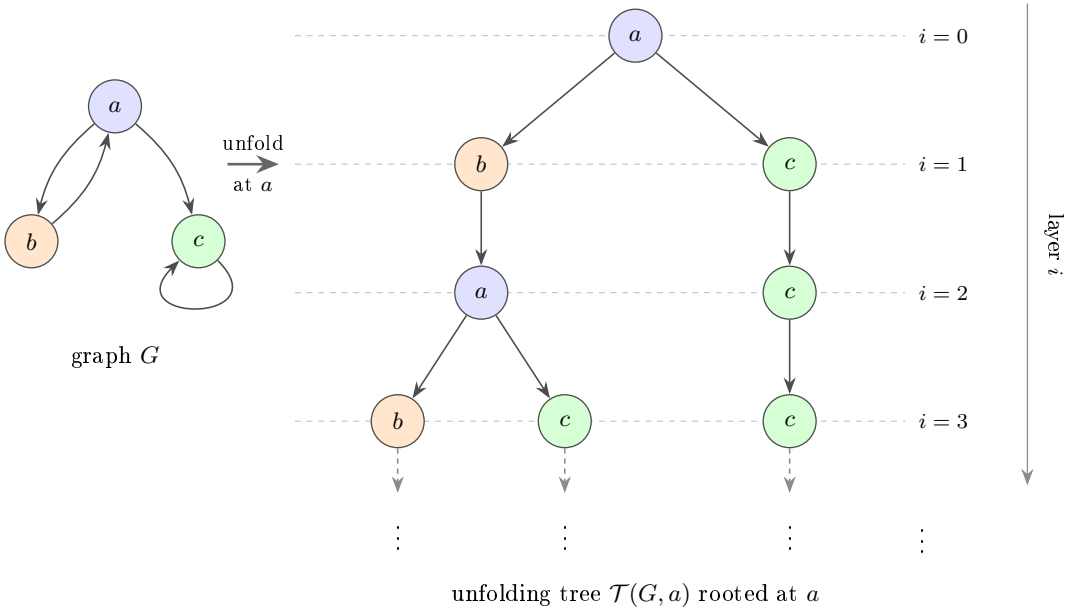


Figure 5.1: Unfolding of a directed graph with cycles. Starting from the root a, every vertex is replaced by one copy per walk reaching it, so the cycle $a \to b \to a$ and the self-loop $c \to c$ are unrolled into an infinite tree; only the layers $i = 0, \ldots, 3$ are drawn. Each tree node at layer i corresponds to a walk of length i from a in G; colours indicate the end vertex of the walk.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Definition 5.1 (Unfolding). Let $G = (V, \mu, L)$ be a labeled multidigraph and $u \in V$. The *unfolding* $\mathcal{T}(G, u)$ is the rooted, labeled, unordered tree defined as follows. The nodes of $\mathcal{T}(G, u)$ are the walks in G starting at u, i.e. sequences $w = (x_0, e_1, x_1, \ldots, e_k, x_k)$ of vertices $x_j \in V$ of G with $x_0 = u$ and $e_j \in \{1, \ldots, \mu(x_{j-1}, x_j)\}$ selecting one of the parallel edges from x_{j-1} to x_j. Each walk, taken as a whole, is one tree node; the same vertex of G can therefore occur in many tree nodes. The root is the walk (u) of length 0, the parent of a walk of length $k \geq 1$ is obtained by deleting its last step (e_k, x_k), and the label of w is $L(x_k)$, the label of the vertex at which the walk ends. The *truncation* $\mathcal{T}_k(G, u)$ is the subtree of walks of length at most k.

Isomorphism of rooted labeled unordered trees means a label-preserving bijection preserving the root and the parent relation. Two finite such trees are isomorphic if and only if the roots have equal labels and there is a bijection between the children of the two roots that maps each child subtree to an isomorphic child subtree. We use two elementary facts. First, the subtree of $\mathcal{T}(G, u)$ below a walk ending at x is isomorphic to $\mathcal{T}(G, x)$. Second, the root of $\mathcal{T}(G, u)$ has exactly $\mu(u, x)$ children whose subtrees are copies of $\mathcal{T}(G, x)$, for each x. Unfoldings are infinite as soon as a cycle is reachable, but they are finitely branching. Figure 5.1 shows an example. For nodes u of G_1 and v of G_2, which may be the same graph, we write $u \equiv v$ if $\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$.

LEMMA 5.2 (Truncations suffice). *$\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$ if and only if $\mathcal{T}_k(G_1, u) \cong \mathcal{T}_k(G_2, v)$ for every $k \geq 0$.*

Proof. ($\Rightarrow$) An isomorphism preserves depth, so it restricts to isomorphisms of the truncations. ($\Leftarrow$) Let I_k be the set of isomorphisms $\mathcal{T}_k(G_1, u) \to \mathcal{T}_k(G_2, v)$. Each I_k is finite (the trees are finite) and nonempty by assumption, and restriction maps I_{k+1} into I_k. Let $J_k \subseteq I_k$ be the set of elements of I_k that are restrictions of elements of I_j for every $j > k$. Since the images of I_j in I_k form a decreasing chain of finite nonempty sets, J_k is nonempty, and every element of J_k is the restriction of an element of J_{k+1}. By Kőnig's lemma [24] (or simply by induction) there is a sequence $\phi_k \in J_k$ with ϕ_{k+1} extending ϕ_k. Their union is an isomorphism $\mathcal{T}(G_1, u) \to \mathcal{T}(G_2, v)$. ∎

5.2 Stable partitions and the unfolding quotient. For a node u and a set Y of nodes of a labeled multidigraph $G = (V, \mu, L)$, let $\mu(u, Y) = \sum_{y \in Y} \mu(u, y)$ be the number of edges from u into Y. A partition P of V is *stable with respect to* Y if $\mu(u, Y) = \mu(u', Y)$ for any two nodes u, u' in the same block of P, and P is *stable* if the nodes in each block have equal labels and P is stable with respect to each of its blocks. Stability with respect to Y is preserved when P is refined, and stability with respect to D and to a subset $E \subseteq D$ implies stability with respect to $D \setminus E$, since $\mu(u, D \setminus E) = \mu(u, D) - \mu(u, E)$.

LEMMA 5.3 (Stable partitions). *The classes of $\equiv$ on V form a stable partition, and every stable partition of V refines it. Hence the classes of $\equiv$ form the coarsest stable partition of V.*

Proof. If $u \equiv u'$, an isomorphism $\mathcal{T}(G, u) \to \mathcal{T}(G, u')$ preserves the label of the root and maps the children of the root bijectively onto the children of the root, and the subtree below each child, which is the unfolding of the end vertex of the corresponding edge, onto an isomorphic subtree. Hence $L(u) = L(u')$, and u and u' have the same number of edges into every class of $\equiv$.

Conversely, let P be a stable partition. We show by induction on k that $\mathcal{T}_k(G, u) \cong \mathcal{T}_k(G, u')$ for any two nodes u, u' in the same block of P. For $k = 0$ this holds because their labels are equal. For the step, u and u' have the same number of edges into every block B of P. Bijections between these edges, for all blocks B, give a bijection between the children of the roots of $\mathcal{T}_{k+1}(G, u)$ and $\mathcal{T}_{k+1}(G, u')$ in which corresponding children are walks ending in the same block, and their depth-k subtrees are isomorphic by the induction hypothesis. Hence $\mathcal{T}_{k+1}(G, u) \cong \mathcal{T}_{k+1}(G, u')$, and $u \equiv u'$ by Lemma 5.2. ∎

We write $[u]$ for the class of u, and $C : V \to V/{\equiv}$ for the map $u \mapsto [u]$.

Definition 5.4 (Unfolding quotient). We define the quotient multidigraph $Q = G/{\equiv}$, written (V_Q, μ_Q, L_Q), as follows: its vertices are the classes of $\equiv$, its labels are $L_Q([u]) = L(u)$, and its multiplicities are

$$\mu_Q(A, B) = \sum_{x \in B} \mu(u, x) \quad \text{for any } u \in A,$$

i.e. the number of edges from a single representative of A into B.

This is *not* the naive quotient that creates one edge $[u] \to [v]$ for every edge (u, v) of G: that construction multiplies multiplicities by class sizes and does not preserve unfoldings. Definition 5.4 coincides with the minimum

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

base of a graph fibration [10]. We call a labeled multidigraph *reduced* if distinct vertices have non-isomorphic unfoldings.

LEMMA 5.5. *The multiplicities μ_Q do not depend on the chosen representatives, and $\mathcal{T}(Q,[u]) \cong \mathcal{T}(G,u)$ for every $u \in V$. The quotient Q is reduced, so its coarsest stable partition is discrete. Finally, C maps every edge of G to an edge of Q, so Q is strongly connected if G is.*

Proof. The first statement is the stability of $\equiv$ (Lemma 5.3). Next we show $\mathcal{T}_k(Q,[u]) \cong \mathcal{T}_k(G,u)$ by induction on k, simultaneously for all u. The case $k = 0$ holds by the choice of L_Q. For the step, the root of $\mathcal{T}_{k+1}(G,u)$ has, for each class B, exactly $\mu_Q([u],B)$ children whose subtrees are $\mathcal{T}_k(G,x)$ with $x \in B$. By the induction hypothesis each of these is isomorphic to $\mathcal{T}_k(Q,B)$, and the root of $\mathcal{T}_{k+1}(Q,[u])$ has exactly $\mu_Q([u],B)$ children with subtree $\mathcal{T}_k(Q,B)$. Lemma 5.2 gives the full unfoldings. If $A \neq B$ in V_Q with representatives $u \not\equiv v$, then $\mathcal{T}(Q,A) \cong \mathcal{T}(G,u) \ncong \mathcal{T}(G,v) \cong \mathcal{T}(Q,B)$, so Q is reduced, and by Lemma 5.3 applied to Q its coarsest stable partition is discrete. Finally, for an edge (x,y) of G, choosing x as the representative of $[x]$ in Definition 5.4 shows that $\mu_Q([x],[y]) \geq \mu(x,y) > 0$. So C maps walks of G to walks of Q, and since C is surjective, Q is strongly connected if G is. ∎

In reduced graphs, isomorphic unfoldings come from isomorphic reachable subgraphs.

LEMMA 5.6 (Reduced graphs). *Let Q_1 and Q_2 be reduced labeled multidigraphs, $A \in V_{Q_1}$ and $B \in V_{Q_2}$. Then $A \equiv B$ if and only if there is an isomorphism between the sub-multigraphs induced on $R(A)$ and $R(B)$ that maps A to B.*

Proof. ($\Leftarrow$) The unfolding of a vertex depends only on the sub-multigraph induced on its reachable set, and isomorphisms preserve unfoldings.

($\Rightarrow$) Let $\tau : \mathcal{T}(Q_1,A) \to \mathcal{T}(Q_2,B)$ be an isomorphism. Define $g : R(A) \to R(B)$ by letting $g(X)$ be the end vertex of $\tau(w)$ for any walk w from A to X. The subtree below w is $\mathcal{T}(Q_1,X)$, and the subtree below $\tau(w)$ is $\mathcal{T}(Q_2,g(X))$, so $X \equiv g(X)$. Since Q_2 is reduced, $g(X)$ does not depend on the choice of w. The same argument with τ^{-1} gives an inverse map, so g is a bijection with $g(A) = B$, and it preserves labels. For multiplicities, fix X and Z in $R(A)$. Since Q_1 is reduced, the root of $\mathcal{T}(Q_1,X)$ has exactly $\mu_{Q_1}(X,Z)$ children whose subtrees are isomorphic to $\mathcal{T}(Q_1,Z)$. Since Q_2 is reduced and $Z \equiv g(Z)$, the root of $\mathcal{T}(Q_2,g(X))$ has exactly $\mu_{Q_2}(g(X),g(Z))$ children of this isomorphism type. An isomorphism between $\mathcal{T}(Q_1,X)$ and $\mathcal{T}(Q_2,g(X))$ preserves the number of children of each isomorphism type, so $\mu_{Q_1}(X,Z) = \mu_{Q_2}(g(X),g(Z))$. Thus g is an isomorphism. ∎

5.3 Canonical refinement. The coarsest stable partition can be computed by partition refinement [13, 35]. We need a version whose result does not depend on how the nodes are named, so that its block numbers can serve as a canonical order. This is the *canonical colour refinement* of Berkholz, Bonsma and Grohe [8]. It maintains a partition into numbered blocks and a list of blocks to refine with. Refining with a block E splits every block into groups of nodes with equal numbers of edges into E. The group with the smallest number keeps the number of the block, and the other groups receive new numbers in increasing order of their number of edges. As in Hopcroft's algorithm [17], a split block that is not in the list adds all of its groups to the list except a largest one, which gives the running time. Every decision depends only on block numbers, block sizes and numbers of edges, never on the names of the nodes. Our stable partitions are the stable colourings of [8], which refine along outgoing edges, and for multidigraphs they are the edge-colour-stable colourings with a single edge colour, in which parallel edges are counted.

LEMMA 5.7 (Canonical refinement [8]). *Let G be a labeled multidigraph with n nodes and m edges, and start canonical colour refinement from the partition into classes of nodes with equal labels, numbered in increasing order of the encodings of their labels. It computes the coarsest stable partition, whose blocks are the classes of $\equiv$, in $O((n+m)\log(n+m))$ time plus the time to sort the labels. Its block numbers are canonical: if $\psi : G_1 \to G_2$ is an isomorphism, then for every node x of G_1, the block of $\psi(x)$ in the result for G_2 has the same number as the block of x in the result for G_1. In particular, if the coarsest stable partition of G is discrete, the block numbers form a canonical order of V.*

Proof. Berkholz, Bonsma and Grohe show that their algorithm computes, within this time, a colouring whose partition is the coarsest stable partition that refines the initial one, for digraphs and for edge-coloured digraphs with parallel edges. They also show that it is canonical: every isomorphism that preserves the initial colouring

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 5.1 Local Component Hash

```
function LOCAL_COMP_HASH(S = (V_S, μ_S, L_S))                    ▷ S strongly connected
    C ← the classes of ≡ on V_S                                  ▷ partition refinement, Lemma 5.7
    Q = (V_Q, μ_Q, L_Q) ← S/C                                    ▷ Definition 5.4; C : V_S → V_Q maps nodes to classes
    κ ← the block numbers of the canonical refinement of Q       ▷ a canonical order, Lemma 5.8
    labels ← [L_Q(X) for X ∈ V_Q in increasing order of κ(X)]
    edges ← SORTED([(κ(X), κ(Y), μ_Q(X, Y)) | X, Y ∈ V_Q, μ_Q(X, Y) > 0])
    g ← H(enc((labels, edges)))
    return {x ↦ H(enc((κ(C(x)), g))) | x ∈ V_S}
end function
```

preserves the computed colours. A label-preserving isomorphism preserves the initial numbering, which depends only on the labels. By Lemma 5.3, the blocks of the coarsest stable partition are the classes of $\equiv$. ∎

We assume from now on that m is bounded by a polynomial in n, which holds for digraphs. Then $\log(n+m) = O(\log n)$, and refinement takes $O((n+m)\log n)$ time.

5.4 Hashing a strongly connected graph. In a strongly connected graph, every node reaches every other node, so a canonical order of the whole quotient depends only on what each node can observe. Algorithm 5.1 serializes the quotient once in this order, and hashes each node by its position.

LEMMA 5.8. *Let S be strongly connected and let Q be its quotient. Then Q is strongly connected, κ is injective, and Q has no automorphism other than the identity.*

Proof. By Lemma 5.5, Q is strongly connected and its coarsest stable partition is discrete, so κ is injective by Lemma 5.7. An automorphism of Q maps every vertex to a vertex with an isomorphic unfolding, so it is the identity because Q is reduced. ∎

In particular, no orbit computation is needed: the canonical index of a vertex of Q identifies it uniquely.

THEOREM 5.9 (Correctness of the local component hash). *Let S_1, S_2 be strongly connected labeled multidigraphs, let $x_1 \in V_{S_1}$ and $x_2 \in V_{S_2}$, and write $h_S = \text{LOCAL_COMP_HASH}(S)$. Assuming $\mathcal{H}$ is injective, $h_{S_1}(x_1) = h_{S_2}(x_2)$ if and only if $x_1 \equiv x_2$. Algorithm 5.1 runs in $O((n+m)\log n)$ time, where n and m are the numbers of vertices and edges of S, plus the time to sort the labels and to hash.*

Proof. Let Q_1, Q_2 be the quotients, $A = C_1(x_1)$ and $B = C_2(x_2)$.

($\Leftarrow$) By Lemma 5.5, $A \equiv B$. By Lemmas 5.6 and 5.8, the reachable sets of A and B are all of Q_1 and Q_2, and there is an isomorphism $\psi : Q_1 \to Q_2$ with $\psi(A) = B$. By Lemma 5.7, $\kappa_2(\psi(X)) = \kappa_1(X)$ for every vertex X of Q_1. Hence the label lists, the edge lists, the hashes g and the indices of A and B coincide.

($\Rightarrow$) By injectivity, $\kappa_1(A) = \kappa_2(B)$, and the label and edge lists coincide. Since κ_1 and κ_2 are injective, mapping the vertex with $\kappa_1 = i$ to the vertex with $\kappa_2 = i$, for every i, is an isomorphism $Q_1 \to Q_2$ that maps A to B. Hence $A \equiv B$, and $x_1 \equiv x_2$ by Lemma 5.5.

The running time follows from Lemma 5.7, applied to S and to Q, together with $O(n+m)$ time for the quotient and $O(m \log n)$ time for sorting the edges. ∎

5.5 The local Merkle hash. Isomorphism of unfoldings is a component equivalence, and by Theorem 5.9 the local component hash realizes it. In the *local variant* $\mathcal{H}_M^{\text{loc}}$ of the Merkle hash, Algorithm 4.1 uses the local component hash, and we write $\approx_{\text{loc}}$ for the Merkle equivalence of isomorphism of unfoldings. Theorem 4.2 gives the following.

COROLLARY 5.10. *Assuming $\mathcal{H}$ is injective, $\mathcal{H}_M^{\text{loc}}$ realizes $\approx_{\text{loc}}$. It runs in $O((n+m)\log n)$ time for a graph with n nodes and m edges, plus the cost of hashing.*

Proof. The first statement is Theorem 4.2. A component S with n_S nodes and m_S internal edges is hashed in $O((n_S + m_S)\log n_S)$ time by Theorem 5.9. The components are disjoint, and every edge is internal to one component or leaves one, so with the complexity paragraph of Section 4 the total is $O((n+m)\log n)$. ∎

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

The local equivalence has a direct description in terms of unfoldings. The *SCC-annotated unfolding* $\mathcal{T}^{\mathrm{scc}}(G,u)$ is the unfolding $\mathcal{T}(G,u)$ in which every tree edge, corresponding to a step from x_{k-1} to x_k, is marked *internal* if $x_k \in \mathrm{scc}(x_{k-1})$ and *external* otherwise. Isomorphisms of annotated unfoldings must preserve the marks.

PROPOSITION 5.11. *For nodes u of G_1 and v of G_2, $u \approx_{\mathrm{loc}} v$ if and only if $\mathcal{T}^{\mathrm{scc}}(G_1,u) \cong \mathcal{T}^{\mathrm{scc}}(G_2,v)$.*

Proof. Marks depend only on the endpoints of each step, so the subtree of $\mathcal{T}^{\mathrm{scc}}(G,u)$ below a walk ending at y is $\mathcal{T}^{\mathrm{scc}}(G,y)$. The walks from u that use only internal steps stay in $\mathrm{scc}(u)$. They form a subtree, the *internal part*, which is the unfolding at u of the subgraph induced on $\mathrm{scc}(u)$. A tree node of the internal part ending at x has one internal child for each internal edge of x and one external child for each external edge. We proceed by induction on $\max(\mathrm{ht}(u),\mathrm{ht}(v))$.

($\Rightarrow$) Let τ be an isomorphism of the unfoldings of $C^{\approx}(u)$ and $C^{\approx}(v)$, i.e. of the internal parts, preserving $L^{\approx}$. For a tree node w ending at x with $\tau(w)$ ending at x', we have $L(x) = L(x')$ and $X^{\approx}(x) = X^{\approx}(x')$. So there is a bijection between the external successors of x and those of x' relating only $\approx_{\mathrm{loc}}$-equivalent nodes, whose annotated unfoldings are isomorphic by the induction hypothesis. Extending τ at every external child by these isomorphisms gives an isomorphism $\mathcal{T}^{\mathrm{scc}}(G_1,u) \to \mathcal{T}^{\mathrm{scc}}(G_2,v)$.

($\Leftarrow$) An isomorphism σ of the annotated unfoldings maps internal children to internal children, so it restricts to an isomorphism τ of the internal parts preserving L. For a tree node w ending at x with $\sigma(w)$ ending at x', σ maps the external children of w bijectively onto those of $\sigma(w)$, with isomorphic annotated subtrees. By the induction hypothesis the corresponding external successors are $\approx_{\mathrm{loc}}$-equivalent, so $X^{\approx}(x) = X^{\approx}(x')$. Thus τ preserves $L^{\approx}$, and $u \approx_{\mathrm{loc}} v$. ∎

PROPOSITION 5.12. *For nodes u of G_1 and v of G_2,*

$$(G_1[R(u)],u) \cong (G_2[R(v)],v) \;\Rightarrow\; u \approx_{\mathrm{glob}} v \;\Rightarrow\; u \approx_{\mathrm{loc}} v \;\Rightarrow\; \mathcal{T}(G_1,u) \cong \mathcal{T}(G_2,v),$$

and none of the converse implications holds.

Proof. A rooted isomorphism of reachable subgraphs maps SCCs to SCCs, so it witnesses Definition 4.1 for $\approx_{\mathrm{glob}}$ at every level. For the second implication, proceed by induction on height. If ϕ is a rooted isomorphism witnessing $u \approx_{\mathrm{glob}} v$, then ϕ preserves the labels $L^{\approx}$ defined with $\approx_{\mathrm{loc}}$ as well. This holds because, by the induction hypothesis, a matching of external successors by $\approx_{\mathrm{glob}}$-equivalent nodes is also a matching by $\approx_{\mathrm{loc}}$-equivalent nodes. A rooted isomorphism induces an isomorphism of unfoldings, so $u \approx_{\mathrm{loc}} v$. The third implication follows from Proposition 5.11 by forgetting the marks.

For strictness, all nodes in the following examples carry the same label.

- *First implication.* Let $x \to p$, $x \to q$, $p \to z$ and $q \to z$, and compare with the same graph in which z is replaced by two copies z_1, z_2 with $p \to z_1$ and $q \to z_2$. The nodes x are $\approx_{\mathrm{glob}}$-equivalent, since all SCCs are singletons and the external multisets agree, but the reachable subgraphs have 4 and 5 nodes.
- *Second implication.* A node with a self-loop and a node on a 2-cycle are $\approx_{\mathrm{loc}}$-equivalent, since both internal unfoldings are infinite paths without external edges. They are not $\approx_{\mathrm{glob}}$-equivalent, since their SCCs have different sizes.
- *Third implication.* A node u with a self-loop, and a node v whose only edge leads to a node w with a self-loop, have isomorphic unfoldings (infinite paths). They are not $\approx_{\mathrm{loc}}$-equivalent, because the first step from u is internal and the first step from v is external. ∎

Remark 5.13 (Why the internal/external marks remain). In both variants, a node x that lies on no cycle is hashed as $F(L(x), \text{successor hashes})$ for a function F built from $\mathcal{H}$. No scheme of this kind can realize plain unfolding isomorphism. In the third example above, unfolding isomorphism would require $h(v) = h(w)$, while $h(v) = F(L, \{\!\{h(w)\}\!\})$. So computing $h(w)$ would produce a fixed point of $y \mapsto F(L, \{\!\{y\}\!\})$, which is infeasible if $\mathcal{H}$ behaves like a random function. The marks are thus the price of Merkle-style compositionality over the condensation of the graph itself. The local hash of Section 5.6 realizes plain unfolding isomorphism by applying the same recursion to the condensation of the unfolding quotient instead, where v and w are the same vertex.

The global variant retains the full structure inside each cycle; it suits applications that must distinguish, for example, a single self-referencing node from a cycle of identical nodes. The local variant retains only what can be

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 5.2 Local Digraph Node Hash

```
1: function H_L(G = (V, μ, L))
2:     C ← the classes of ≡ on V                    ▷ partition refinement, Lemma 5.7
3:     Q = (V_Q, μ_Q, L_Q) ← G/C                    ▷ Definition 5.4
4:     h ← H_M(Q, LOCAL_COMP_HASH)                  ▷ Algorithm 4.1, i.e. h = H_M^loc(Q)
5:     return {u ↦ h[C(u)] | u ∈ V}
6: end function
```

observed by following edges, together with whether each edge leaves its component. It runs in $O((n+m)\log n)$ time in every case, while the global variant can be much slower (Section 9), it merges nodes that no traversal can tell apart, and it is the variant used in Section 7.

5.6 Hashing all nodes. If a graph is not strongly connected, the canonical order of its whole quotient also depends on vertices that a node cannot reach, so the hash of the node would not be determined by its unfolding. The local Merkle hash avoids this, but by Remark 5.13 it also distinguishes nodes by the marks of their annotated unfoldings. We therefore apply it to the quotient, where the marks make no difference (Corollary 5.14). Algorithm 5.2 gives the pseudo-code.

COROLLARY 5.14. *For vertices A and B of unfolding quotients Q_1 and Q_2, the following are equivalent: $A \equiv B$; $A \approx_{\mathrm{glob}} B$; and $A \approx_{\mathrm{loc}} B$. In particular, $\mathcal{H}_M^{\mathrm{loc}}(Q)$ realizes unfolding isomorphism, and so does $\mathcal{H}_M^{\mathrm{glob}}(Q)$ if Q has no parallel edges.*

Proof. If $A \equiv B$, Lemmas 5.5 and 5.6 give an isomorphism between the sub-multigraphs induced on $R(A)$ and $R(B)$ that maps A to B, so $A \approx_{\mathrm{glob}} B$ and $A \approx_{\mathrm{loc}} B$ by Proposition 5.12. Conversely, $A \approx_{\mathrm{glob}} B$ implies $A \approx_{\mathrm{loc}} B$, which implies $A \equiv B$ by the same proposition. The last statement follows from Corollaries 4.3 and 5.10. ∎

THEOREM 5.15 (Correctness of the local hash). *$\mathcal{H}_L$ realizes unfolding isomorphism: assuming $\mathcal{H}$ is injective, for nodes u of G_1 and v of G_2, $\mathcal{H}_L(G_1)[u] = \mathcal{H}_L(G_2)[v]$ if and only if $\mathcal{T}(G_1, u) \cong \mathcal{T}(G_2, v)$. Algorithm 5.2 runs in $O((n+m)\log n)$ time for a graph with n nodes and m edges, plus the time to sort labels and to hash strings of total length $O(n+m)$ labels and hashes.*

Proof. Let Q_1, Q_2 be the quotients. By Lemma 5.5, $\mathcal{T}(G_i, u) \cong \mathcal{T}(Q_i, C_i(u))$, and $\mathcal{H}_L(G_i)[u] = \mathcal{H}_M^{\mathrm{loc}}(Q_i)[C_i(u)]$. By Corollary 5.10, $\mathcal{H}_M^{\mathrm{loc}}(Q_1)[A] = \mathcal{H}_M^{\mathrm{loc}}(Q_2)[B]$ if and only if $A \approx_{\mathrm{loc}} B$, which holds if and only if $A \equiv B$ by Corollary 5.14.

Running time. Line 2 takes $O((n+m)\log n)$ time by Lemma 5.7. The quotient has at most n vertices and m edges, and it is computed in $O(n+m)$ time. Line 4 takes $O((n+m)\log n)$ time by Corollary 5.10. Every relabeled label is serialized once with its component, and every node hash has constant size, which bounds the total length of the hashed strings. ∎

5.7 Alternative formulations of local hashing. The local hash realizes what an observer can see by traversing the graph forward along its directed edges. We note two other formulations.

Walk languages. Treat labels as letters of an alphabet and define the *vertex language* of u as the set of label sequences of walks starting at u:

$$\mathrm{Lang}_G(u) = \{L(u)\} \cup \bigcup_{v \in \mathrm{succ}(u)} \{L(u) \circ x \mid x \in \mathrm{Lang}_G(v)\}. \tag{5.1}$$

This resembles a nondeterministic finite automaton in which letters are emitted when visiting nodes rather than when traversing edges, and in which every state is accepting. Every vertex language is therefore prefix-closed.

PROPOSITION 5.16. *Deciding $\mathrm{Lang}_G(u) = \mathrm{Lang}_G(v)$ is PSPACE-complete. Consequently, unless* P = PSPACE, *there is no polynomial-time computable function h with $h(u) = h(v)$ if and only if $\mathrm{Lang}_G(u) = \mathrm{Lang}_G(v)$.*

Proof. Membership: a vertex language is the language of an NFA with one state per vertex plus an initial state, and NFA equivalence is in PSPACE. Hardness: we reduce from equivalence of NFAs in which all states are final, which is PSPACE-complete over alphabets of size at least two [22]. Let $M = (P, \Sigma, \delta, p_0, P)$ be such an NFA without ε-moves. Build a vertex-labeled graph with a root r labeled by a fresh letter # and one vertex t labeled

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

a for each transition $t = (p, a, q) \in \delta$. Add edges $r \to t$ for every transition leaving p_0, and $t \to t'$ whenever $t = (p, a, q)$ and t' leaves q. Walks from r correspond exactly to runs of M from p_0, and every run is accepting, so $\mathrm{Lang}(r) = \{\#w \mid w \in L(M)\}$. The graph has $O(|\delta|^2)$ edges. Building the graphs for two such NFAs M_1, M_2 side by side gives roots with equal languages if and only if $L(M_1) = L(M_2)$. The final statement follows because h would decide the problem in polynomial time. ∎

Bisimulation. Nodes a and b (possibly in different graphs) are *bisimilar* if they are related by the largest relation $\mathcal{R}$ such that $a\mathcal{R}b$ implies: (a) $L(a) = L(b)$; (b) for every edge $a \to a'$ there is an edge $b \to b'$ with $a'\mathcal{R}b'$; and (c) for every edge $b \to b'$ there is an edge $a \to a'$ with $a'\mathcal{R}b'$ [38]. The existential quantifiers, in place of a bijection between successors, mean that bisimulation ignores duplicated bisimilar successors: bisimilar nodes may have different out-degrees. Unfolding isomorphism is the counting analogue of bisimilarity and implies it.

A partition of V is *bisimulation-stable* if the nodes in each block have equal labels and, for any two blocks B and B', either every node of B has a successor in B' or none has. The bisimilarity classes form the coarsest bisimulation-stable partition: by clauses (b) and (c) bisimilar nodes have successors in the same classes, and conversely, the relation of lying in the same block of a bisimulation-stable partition is a bisimulation. The relational coarsest partition algorithm of Paige and Tarjan [35] computes these classes in $O(m \log n)$ time, see also [21]. The *bisimulation quotient* has the classes as vertices, with the labels of their members, and an edge $A \to B$ if and only if some, equivalently every, node of A has a successor in B. The pairs $(u, [u])$ form a bisimulation, so every node is bisimilar to its class, and by transitivity distinct vertices of a bisimulation quotient are not bisimilar.

The *bisimulation hash* $\mathcal{H}_B$ is Algorithm 5.2 with three changes, which apply to Algorithm 5.1 as well. The classes are the bisimilarity classes, and the quotients are bisimulation quotients. Algorithm 4.1 is used in its set variant, in which the external hashes $X(x)$ form a set rather than a multiset. And the edges are serialized without multiplicities. The canonical order κ is still computed by the canonical colour refinement of Section 5.3: unfolding isomorphism implies bisimilarity, so no two vertices of a bisimulation quotient have isomorphic unfoldings, and its coarsest stable partition is discrete.

THEOREM 5.17. *$\mathcal{H}_B$ realizes bisimilarity: assuming $\mathcal{H}$ is injective, $\mathcal{H}_B(G_1)[u] = \mathcal{H}_B(G_2)[v]$ if and only if u and v are bisimilar. It runs in $O((n+m)\log n)$ time, plus the cost of hashing.*

Proof. We write $\sim$ for bisimilarity, and follow the proofs of Lemma 5.6, Theorem 5.9 and Corollary 5.14.

Bisimulation quotients. Let A and B be vertices of bisimulation quotients Q_1 and Q_2. Then $A \sim B$ if and only if there is an isomorphism between the sub-digraphs induced on $R(A)$ and $R(B)$ that maps A to B. Indeed, such an isomorphism is a bisimulation. Conversely, if $A \sim B$, then by induction along walks, using clauses (b) and (c), every vertex of $R(A)$ is bisimilar to a vertex of $R(B)$, and vice versa. Since distinct vertices of the same quotient are not bisimilar, $\sim$ relates $R(A)$ and $R(B)$ by a bijection g with $g(A) = B$, which preserves labels. If $X \to Z$, then $g(X)$ has a successor bisimilar to Z, which can only be $g(Z)$; symmetrically, every edge $g(X) \to g(Z)$ comes from an edge $X \to Z$. So g is an isomorphism.

Strongly connected graphs. As in Lemma 5.5, the bisimulation quotient of a strongly connected graph is strongly connected, and κ is injective on it. With the previous paragraph in place of Lemma 5.6, the proof of Theorem 5.9 shows that the component hashes of x_1 and x_2 are equal if and only if $x_1 \sim x_2$, since an isomorphism of quotients is a bisimulation, and $x_i \sim C_i(x_i)$.

All nodes. Bisimilarity is a component equivalence, and by the previous paragraph the bisimulation component hash realizes it. By Theorem 4.2 for the set variant, $h_1(A) = h_2(B)$ if and only if $A \approx B$, where $\approx$ is the Merkle equivalence of bisimilarity with sets. We show that $A \approx B$ if and only if $A \sim B$, for vertices A of Q_1 and B of Q_2, by induction on $\max(\mathrm{ht}(A), \mathrm{ht}(B))$. Let K and K' be the components of A and B. ($\Leftarrow$) The isomorphism ψ of the first paragraph maps K onto K', and the external successors of every $x \in K$ bijectively onto those of $\psi(x)$. They are bisimilar to their images and have smaller height, so they are $\approx$-equivalent to their images by the induction hypothesis. Hence $X^{\approx}(\psi(x)) = X^{\approx}(x)$, and ψ restricts to an isomorphism $C^{\approx}(A) \to C^{\approx}(B)$, which is a bisimulation relating A and B. So $A \approx B$. ($\Rightarrow$) There is a bisimulation R between $C^{\approx}(A)$ and $C^{\approx}(B)$ that relates A and B and respects the labels $L^{\approx}$. If $x\,R\,x'$, then $L(x) = L(x')$ and $X^{\approx}(x) = X^{\approx}(x')$, so by the induction hypothesis every external successor of x is bisimilar to some external successor of x', and vice versa; and every internal successor of either node is related by R to an internal successor of the other. Hence $R \cup \sim$ is a bisimulation, and $A \sim B$.

Running time. The algorithm of Paige and Tarjan takes $O(m \log n)$ time, and the rest is as in Theorem 5.15. ∎

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

5.8 Relation to prior work. Refinement by counting edges is color refinement, the 1-dimensional Weisfeiler–Leman algorithm [43], applied along outgoing edges; the Weisfeiler–Lehman subtree kernels [40, 39] hash the colors of its rounds. Partition refinement for automata goes back to Hopcroft's DFA minimization algorithm [17, 23]. The equivalence $\equiv$ and the quotient Q correspond to the fibration-theoretic notions of *views* and *minimum bases* [44, 10], and for undirected graphs, isomorphism of unfoldings is isomorphism of universal covers [32]. The canonical colour refinement of Berkholz, Bonsma and Grohe [8], which we use, computes colourings that are preserved by isomorphisms of the whole graph; colours of nodes in non-isomorphic graphs cannot be compared, which is what the recursion over the components of the quotient achieves. Unfoldings of ordered graphs are regular trees, and unique, maximally shared representations and canonical forms of regular trees and of recursive types have been computed with Hopcroft's algorithm [26, 14, 16]; the ordered variant of Section 6 is closely related to these. To our knowledge, hashes for unordered unfoldings and for bisimilarity that are canonical across graphs and can be computed in near-linear time have not been described before.

6 Outgoing Ordered Edge Graphs

In many real world applications the outgoing edges of each node are ordered. Examples include the links of a web page, the fields of a record, and the arguments of a function call. We take advantage of this extra structure to create simpler and faster hashing algorithms. The algorithms in this section were inspired by the more general Scott graph canonization algorithm of Bloyet, Marteau and Frénod [9]. The basic idea is that, because the edges are ordered, a depth-first search from a given root visits the nodes in an order that is determined by the structure alone.

Model. An *ordered labeled digraph* assigns to each node u a label $L(u)$ and a finite *out-sequence* $s(u) = (s_1(u), \ldots, s_{d(u)}(u))$ of successor nodes; position k is the k-th *port* of u, and repeated entries (two links to the same page) are allowed. Forgetting the order, the entries of $s(u)$ form the multiset $\mathrm{succ}(u)$ of Section 2, so an ordered graph is a labeled multidigraph with additional structure. An *ordered isomorphism* $\phi : G_1 \to G_2$ is a label-preserving bijection with $s(\phi(u)) = (\phi(s_1(u)), \ldots, \phi(s_{d(u)}(u)))$ for all u. Ordered automorphisms, orbits and rooted ordered isomorphisms are defined accordingly. For a node n we write $G[R(n)]$ for the ordered subgraph induced on its reachable set.

Example 6.1 (Edge order must be serialized). Sorting the relabeled adjacency list, as an earlier version of this algorithm did, loses the port order, and the depth-first numbering does not always recover it because edges to already-visited nodes do not affect the numbering. Take $s(r) = (x, y)$, $s(y) = ()$, and either $s(x) = (y, r)$ or $s(x) = (r, y)$. In both graphs the search from r numbers r, x, y as $0, 1, 2$, and the sorted edge set is $\{(0,1), (0,2), (1,0), (1,2)\}$ in both cases, even though the two graphs are not ordered-isomorphic.

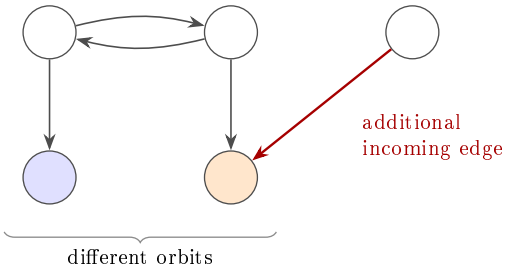


Figure 6.1: The symmetry of the two bottom nodes is broken by an incoming edge from elsewhere in the graph (all nodes carry the same label). Without the highlighted edge the two bottom nodes would be interchangeable; with it, no automorphism maps one onto the other, so they no longer lie in the same orbit. Their reachable subgraphs (a single sink each) are nevertheless identical, so any hash that looks only at reachable subgraphs must give them equal values.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 6.1 Node Hash — Outgoing Ordered Edges

1. From the input node n, perform a depth-first search that explores the out-edges of each node in port order, and number the reachable nodes $0, 1, 2, \ldots$ in order of discovery (preorder).
2. Create the list of original labels in discovery order.
3. For each node in discovery order, write its out-sequence as a sequence of discovery numbers, *keeping the port order* (do not sort).
4. Return $\mathcal{H}_O(G, n) = \mathcal{H}(\text{enc}((\text{label list},\ \text{out-sequence list})))$.

THEOREM 6.2. *$\mathcal{H}_O$ realizes rooted ordered isomorphism of reachable subgraphs: assuming $\mathcal{H}$ is injective, $\mathcal{H}_O(G_1, n_1) = \mathcal{H}_O(G_2, n_2)$ if and only if there is an ordered isomorphism on the reachable subgraphs $G_1[R(n_1)] \to G_2[R(n_2)]$ mapping n_1 to n_2. It runs in time $O(|R(n)| + |E(R(n))|)$ plus the cost of hashing.*

Proof. ($\Leftarrow$) The depth-first search is deterministic given the port orders. An ordered isomorphism ψ commutes with every step of it: by induction on the steps, the searches from n_1 and $\psi(n_1) = n_2$ discover x and $\psi(x)$ at the same step. Hence ψ preserves discovery numbers, and the serializations coincide. ($\Rightarrow$) Equal serializations have equal lengths. Map the k-th discovered node of G_1 to the k-th discovered node of G_2. The label lists agree, and the out-sequences of corresponding nodes are equal as sequences of discovery numbers, so the map is an ordered isomorphism mapping n_1 (number 0) to n_2. ∎

$\mathcal{H}_O$ is a hash of the reachable subgraph, not of the unfolding. For example, a node with a self-loop and a node on a 2-cycle (with all labels equal) have isomorphic unfoldings but different values of $\mathcal{H}_O$. The unfolding-based notion for ordered graphs uses *ordered unfoldings*, in which the children of every tree node are ordered by port, and isomorphisms must preserve this order; in this section we write $u \equiv v$ if the ordered unfoldings of u and v are isomorphic. An ordered graph is a deterministic automaton whose letters are the ports and whose output at each state is its label and out-degree, and the classes of $\equiv$ are the states of the minimal automaton.

Merkle hashing of ordered graphs. For ordered graphs, Line 5 of Algorithm 4.1 becomes

$$X(x) \leftarrow \big[\,(h[y] \textbf{ if } y \notin S \textbf{ else } \star) \textbf{ for } y \textbf{ in } s(x)\,\big],$$

so that every node x of a component S is relabeled with the sequence whose k-th entry is $h[s_k(x)]$ if $s_k(x) \notin S$ and a *placeholder* $\star$ otherwise, and the positions of internal and external links are both recorded. In Line 8, G' keeps the internal targets of each node in port order, and we encode $\star$ as a value that differs from every hash. Correspondingly, component equivalences are defined on strongly connected ordered graphs, and in Definition 4.1, $X^{\approx}(x)$ is the sequence whose k-th entry is the $\approx$-class of $s_k(x)$ if $s_k(x) \notin \text{scc}(x)$ and $\star$ otherwise, and $C^{\approx}(x)$ keeps the internal targets in port order. Theorem 4.2 holds with the same proof, because L' and the internal targets in port order determine every port, exactly as $C^{\approx}$ does. Rooted ordered isomorphism and isomorphism of ordered unfoldings are component equivalences, and we write $\approx_{\text{glob}}$ and $\approx_{\text{loc}}$ for their Merkle equivalences; in the description of $\approx_{\text{glob}}$ after Corollary 4.3, the external successors are matched port by port, with internal ports mapped by ϕ. Propositions 5.11 and 5.12 hold for ordered graphs with the same proofs and examples, with annotated unfoldings ordered by port and external successors matched port by port. A further example inside a single strongly connected graph is $s(a) = (b, a)$, $s(b) = (a, a)$ with equal labels. Here $a \approx_{\text{loc}} b$, since all nodes have the same label and out-degree, but a and b lie in different orbits, since an automorphism exchanging them would map $s(a) = (b, a)$ to $(a, b) \neq s(b)$.

Section 5 carries over to ordered graphs as follows.

THEOREM 6.3 (Ordered local hashing). *Change Section 5 as follows. A partition is* stable with respect to *a set Y if, for any two nodes u, u' in the same block and every port k, $s_k(u) \in Y$ if and only if $s_k(u') \in Y$; and a stable partition must have equal labels and equal out-degrees in each block. The quotient has the out-sequences $s([u]) = ([s_1(u)], \ldots, [s_{d(u)}(u)])$. The canonical refinement is canonical colour refinement of the edge-coloured digraph in which the edge from u to $s_k(u)$ has colour k [8], started from the classes of nodes with equal label and out-degree. Algorithm 5.1 serializes the out-sequence $(\kappa(s_1(X)), \ldots, \kappa(s_{d(X)}(X)))$ of every vertex X in place of the multiplicities. Finally, Algorithm 5.2 uses the ordered Merkle hash described above. With these*

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Algorithm 6.2 Canonization and Orbit Partitioning for Strongly Connected Ordered Graphs

1. Compute the classes of $\equiv$ with their canonical block numbers by the canonical refinement of Theorem 6.3. Let C be a smallest class, and among the smallest classes the one with the least number.
2. For each $r \in C$, compute $\mathcal{H}_O(G, r)$ using Algorithm 6.1, and let $T \subseteq C$ be the set of nodes with the minimal value. Choose any $r_0 \in T$. Its discovery order is the canonical labeling ρ, and its serialization is the canonical form $\mathrm{canon}(G)$.
3. For $t \in T$, let σ_t map the i-th node discovered from r_0 to the i-th node discovered from t. The orbit of v is $\{\sigma_t(v) \mid t \in T\}$.
4. The graph hash is $\mathcal{H}(\mathrm{enc}(\mathrm{canon}(G)))$, and the orbit index of v is the least value of ρ on the orbit of v. These are used exactly as in Algorithm 3.2.

changes, Lemmas 5.3–5.8, Theorems 5.9 and 5.15, and Corollaries 5.10 and 5.14 hold for ordered unfoldings and ordered isomorphisms. In particular, the local hash realizes isomorphism of ordered unfoldings, and it runs in $O((n+m)\log n)$ time, plus the cost of hashing.

Proof. We list the changes to the proofs. In Lemma 5.3, an isomorphism of ordered unfoldings maps the k-th child of the root to the k-th child of the root, so equivalent nodes have equal out-degrees and equivalent k-th successors; conversely, in the induction, the k-th children of the two roots are matched with each other. In Lemma 5.5, the out-sequences are well defined by stability, and the remaining statements are proved in the same way. In Lemma 5.6, an isomorphism τ of ordered unfoldings maps the walk from A that follows the ports $k_1, \ldots, k_j$ to the walk from B that follows the same ports. Let $g(X)$ be the end vertex of $\tau(w)$ for any walk w from A to X. Since Q_2 is reduced, $g(X)$ does not depend on w, and by symmetry g is a bijection. It preserves labels, and $s_k(g(X)) = g(s_k(X))$, because τ maps the extension of w by port k to the extension of $\tau(w)$ by port k. Lemma 5.7 holds because a partition is stable in the ordered sense if and only if it is edge-colour-stable for these colours: a node u has one edge of colour k into a block Y if $s_k(u) \in Y$, and none otherwise. The proofs of Lemma 5.8 and Theorem 5.9 are unchanged, with out-sequences in place of multiplicities. Corollaries 5.10 and 5.14 and Theorem 5.15 then follow as before from the ordered versions of Theorem 4.2 and Proposition 5.12. □

In practice, the classes can be computed directly as the states of the minimal automaton with the algorithms of Hopcroft and of Valmari and Lehtinen [17, 42], in $O(m \log n)$ time, as the dihash library does.

Strongly connected graphs. For the special case of a strongly connected ordered graph, the node hashes of Algorithm 6.1 yield both a canonical form and the orbit partition. This is the case needed by the global variant of the Merkle hash, which hashes strongly connected components individually. (The local variant uses the ordered version of Algorithm 5.1 instead, which serializes the quotient of each component once.) A search from every node would take $O(n(n+m))$ time. It suffices, however, to search from the nodes of one class of $\equiv$, provided that the class is chosen in a way that isomorphisms respect, and the canonical refinement provides such a choice.

THEOREM 6.4. *Let G_1, G_2 be strongly connected ordered digraphs with n nodes and m edges, and assume $\mathcal{H}$ is injective. Then: (a) $\mathcal{H}_O(G_1, u) = \mathcal{H}_O(G_2, v)$ if and only if there is an ordered isomorphism $\phi : G_1 \to G_2$ with $\phi(u) = v$; (b) $\mathrm{canon}(G_1) = \mathrm{canon}(G_2)$ if and only if $G_1 \cong G_2$, and ρ is an isomorphism onto $\mathrm{canon}(G)$; (c) every ordered automorphism is determined by the image of a single node, so $|\mathrm{Aut}(G)| \le n$ and all orbits have equal size; the maps σ_t of Step 3 are exactly the automorphisms of G, so Step 3 computes the orbits; (d) the node hash of Step 4 realizes rooted ordered isomorphism, as in Theorem 3.2; (e) Algorithm 6.2 runs in $O((n+m)\log n + c\,(n+m))$ time plus the cost of hashing, where $c = |C|$. In particular, it runs in $O((n+m)\log n)$ time if some class of $\equiv$ has a single node, and in $O(n(n+m))$ time in the worst case.*

Proof. (a) Since G_i is strongly connected, $R(u) = V_1$ and $R(v) = V_2$, so this is Theorem 6.2. (b) Let $\phi : G_1 \to G_2$ be an isomorphism. By the canonicity of the block numbers (Lemma 5.7, which holds for ordered graphs by Theorem 6.3), ϕ maps the class of $\equiv$ with number i onto the class with number i, so it preserves the sizes and numbers of the classes, and $\phi(C_1) = C_2$. By (a), ϕ maps nodes of C_1 to nodes of C_2 with equal $\mathcal{H}_O$ values. The minimal values therefore coincide, and all minimizing nodes have the same serialization, which is

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

determined by the hash. Conversely, equal serializations yield an isomorphism as in Theorem 6.2. The discovery order is exactly the bijection onto the serialized graph. (c) An automorphism commutes with the depth-first search from any node u, which reaches all nodes, so it is determined by $\sigma(u)$. Hence $\mathrm{Aut}(G)$ acts semiregularly. For $t \in T$, (a) gives an automorphism that maps r_0 to t. It commutes with the searches, so it maps the i-th node discovered from r_0 to the i-th node discovered from t, and it is σ_t. Conversely, an automorphism σ maps C onto itself by the argument of (b) with $G_1 = G_2$, and $\mathcal{H}_O(G, \sigma(r_0)) = \mathcal{H}_O(G, r_0)$ by (a). Hence $\sigma(r_0) \in T$, and $\sigma = \sigma_{\sigma(r_0)}$ because both are determined by the image of r_0. (d) With (b) in place of the canonizer, the proof of Theorem 3.2 applies verbatim; the orbit index of Step 4 is the minimum canonical index over the orbit by (c). (e) Step 1 takes $O((n+m)\log n)$ time by Theorem 6.3. Step 2 runs c searches of cost $O(n+m)$ each, and Step 3 takes $O(|T|\, n)$ time, where $|T| \le c$. ∎

In the dihash library, Step 1 uses the canonical Hopcroft refinement of the ordered local hash. On the largest strongly connected components of the random graphs of Section 9, about 93% of the nodes are alone in their class of $\equiv$, so $c = 1$ and a single search suffices.

By Theorem 6.4(d), the node hash of Algorithm 6.2 realizes rooted ordered isomorphism. With it as the component hash, the global Merkle hash $\mathcal{H}_M^{\mathrm{glob}}$ of ordered graphs therefore realizes $\approx_{\mathrm{glob}}$ by Theorem 4.2, and a component S costs $O((|S| + m_S)\log|S| + c_S(|S| + m_S))$, where $c_S \le |S|$ is the size of the class C chosen in the relabeled component.

The worst case. The bound of Theorem 6.4(e) is quadratic when every class of $\equiv$ is large, and refining along incoming edges as well does not remove this case. Suppose that all nodes have the same label and the same out-degree, and that for every port k every node is the k-th successor of exactly one node. Then every port permutes the nodes, no refinement along outgoing or incoming edges separates any two nodes, and G is the Schreier graph of the transitive permutation group Γ generated by the ports. The ordered automorphisms of G are the permutations that commute with every port, i.e. the centralizer of Γ in the symmetric group, and the orbit of r is the set of fixed points of the stabilizer of r in Γ. If there is a single port, G is a cycle, and the least rotation of a cyclic string can be computed in linear time [12]. That method uses the fact that every rotation of a string is a substring of the string written twice, which has no analogue for depth-first searches in general graphs. We do not know a subquadratic algorithm for the general case. The canonical code of Jiang and Bunke for ordered undirected graphs, in which the edges at each node are cyclically ordered, is likewise the least of $2m$ codes, one per starting edge, and is computed in $O(m^2)$ time [18, 19].

Why only strongly connected graphs? For a general digraph, the orbit of a node depends not only on the structure of its reachable subgraph but also on its ancestors. Incoming edges can break the symmetry of an orbit, as demonstrated in Figure 6.1, so the reachable-subgraph hashes of Algorithm 6.1 can be equal for nodes in different orbits. In a strongly connected graph this cannot happen, because every node reaches every other node. For general ordered digraphs, canonization is as hard as for digraphs (Proposition 6.5), so a polynomial-time algorithm would also give one for graph isomorphism. The Merkle construction of Section 4 only needs strongly connected components.

PROPOSITION 6.5. *Isomorphism of ordered digraphs is at least as hard as isomorphism of digraphs, even for acyclic ordered digraphs in which every node has out-degree* 0 *or* 2. *A canonization of such ordered digraphs yields a canonization of digraphs with linear overhead.*

Proof. Given a digraph D, possibly with parallel edges, let D' be the ordered digraph with a node for every vertex of D, labeled `v` and with out-degree 0, and a node for every edge $e = (a, b)$ of D, labeled `e` and with out-sequence $s(e) = (a, b)$. It is acyclic, and it is built in linear time. An ordered isomorphism $\phi : D_1' \to D_2'$ preserves labels, so it maps vertices to vertices and edge nodes to edge nodes, and $s(\phi(e)) = (\phi(a), \phi(b))$ for every edge $e = (a, b)$. Hence the restriction of ϕ to the vertices is a bijection that maps the edges from a to b bijectively to the edges from $\phi(a)$ to $\phi(b)$, i.e. an isomorphism $D_1 \to D_2$. Conversely, an isomorphism $\psi : D_1 \to D_2$, together with a bijection between the edges from a to b and those from $\psi(a)$ to $\psi(b)$ for every pair (a, b), is an ordered isomorphism $D_1' \to D_2'$. Thus $D_1 \cong D_2$ if and only if $D_1' \cong D_2'$, and a canonical form of D' is a canonical form of D. ∎

Booth [11] showed that isomorphism of finite automata is polynomial-time equivalent to graph isomorphism.

7 Application Sketch From the algorithms given so far, it may not be obvious how to implement these graph theoretic procedures for real world data. Here we show how to do so for a static hyperlinked web page

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Row 1: Original HTML Pages

index.html

```
<body>
 <h1>Welcome</h1>
 <a href="about.html">
  About Us
 </a>
 <a href="http://e.com">
  External Resource
 </a>
</body>
```

about.html

```
<body>
 <h1>About Us</h1>
 <a href="index.html">
  Home
 </a>
 <a href="contact.html">
  Contact
 </a>
 <a href="http://p.com">
  Partner Site
 </a>
</body>
```

contact.html

```
<body>
 <h1>Contact Us</h1>
 <a href="about.html">
  About
 </a>
 <a href="http://e.com">
  External Resource
 </a>
</body>
```

Row 2: Labels $L(x)$, with a hole {} in place of every link target, and out-sequences $s(x)$

index.html

```
<body>
 <h1>Welcome</h1>
 <a href="{}">
  About Us
 </a>
 <a href="{}">
  External Resource
 </a>
</body>
```

$s =$ (about.html, http://e.com)

about.html

```
<body>
 <h1>About Us</h1>
 <a href="{}">
  Home
 </a>
 <a href="{}">
  Contact
 </a>
 <a href="{}">
  Partner Site
 </a>
</body>
```

$s =$ (index.html, contact.html, http://p.com)

contact.html

```
<body>
 <h1>Contact Us</h1>
 <a href="{}">
  About
 </a>
 <a href="{}">
  External Resource
 </a>
</body>
```

$s =$ (about.html, http://e.com)

Row 3: Relabeled component G' of Algorithm 4.1 for ordered graphs, with labels $L'(x) = (L(x), X(x))$

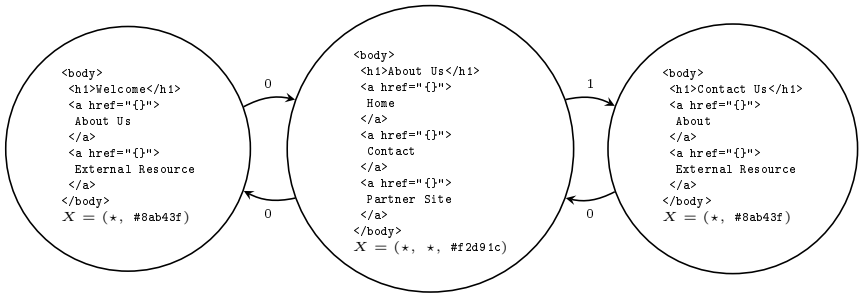


Figure 7.1: Hashing a three page website that forms a single SCC. In Row 3, $X(x)$ has one entry per hole: the placeholder $\star$ for a link inside the SCC, and the recursively computed hash of the target for a link leaving it (#8ab43f and #f2d91c are hashes of external pages). The edges are the internal links, numbered in port order, so the j-th $\star$ of a page is its internal link numbered $j-1$. Filling the holes from $X(x)$ and the edges recovers every link.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

system stored in a content addressed distributed hash table (DHT).

In this sketch, we assume that web pages are static HTML files that link to each other and to external pages, and that external pages are themselves published in the same content-addressed way. (An external page that links back into our site would belong to the same SCC and be hashed together with it.) The pages form an ordered digraph in which each page is a node, the links are edges, and the out-edges of a node are ordered by their position in the HTML file. Figure 7.1 gives a concrete example of the process described here.

The label $L(x)$ of a page is its HTML text with every link target replaced by a hole, and its out-sequence $s(x)$ lists the link targets in the order in which they appear (Row 2 of Figure 7.1). We compute the SCCs of the link graph and process them sinks first with Algorithm 4.1, in its version for ordered graphs (Section 6). Each page is relabeled with $L'(x) = (L(x), X(x))$. Here $X(x)$ has one entry per hole: the placeholder $\star$ for a link to a page of the same SCC, and the recursively computed hash of the target for a link leaving the SCC (Row 3). Because $L'(x)$ is not hashed, the page text and its external links appear verbatim in it, and its internal links are the edges of G' in port order. We use the local Merkle hash of Section 5.5, in its ordered version (Section 6). For each SCC, Algorithm 5.1 computes the ordered quotient, which merges pages that cannot be told apart by following links, together with its canonical order. We then insert the following items into the distributed hash table:

- The strongly connected component as a whole:
  - **Value**: the pair (labels, edges) of Algorithm 5.1, i.e. the labels L' of the quotient in canonical order, together with the internal out-sequence of each quotient node in canonical indices.
  - **Key**: the string hash $\mathcal{H}$ of the value, which is exactly the component hash g of Algorithm 5.1.
- For each page x, a reference to a specific node of the quotient:
  - **Value**: the pair $(\text{index}[C(x)], g)$ of the page's canonical index and the key of its SCC.
  - **Key**: the string hash $\mathcal{H}$ of the value, which is exactly the Merkle hash $h[x]$ of the page.

By Theorem 4.2 and Proposition 5.11, two pages receive the same key exactly when their SCC-annotated ordered unfoldings are isomorphic. In other words, no sequence of clicks can tell them apart, except by observing which links leave their SCC. In particular, pages in the same orbit share a key, and so do duplicated pages. This is intended: the store is content-addressed, and such pages are indistinguishable by navigation. No orbit computation is needed, because quotients of strongly connected graphs have no non-trivial automorphisms (Lemma 5.8). The whole site is hashed in near-linear time in practice (Section 9). If instead the exact page structure of every SCC must be preserved, the global variant can be used: Algorithm 6.2 replaces the quotient by the full SCC with its orbit partition, and only pages in the same orbit share a key.

Suppose that a user wants to navigate our website and has obtained the hash of one specific page. The browser retrieves the pair (i, g) stored under this hash, and then the value stored under g, which describes the whole quotient. To display quotient node i, the browser fills the holes of its label $L(x)$ in order. A hole whose entry in $X(x)$ is a hash receives that hash, which is the key of an external page. The j-th hole whose entry is $\star$ receives the j-th internal edge of node i, which is the canonical index of a node of the same quotient. If the user clicks a link within the same SCC, the browser simply displays the corresponding node of the quotient it already holds. Since the quotient preserves ordered unfoldings (Lemma 5.5 and Theorem 6.3), the user sees exactly the sequence of pages they would see on the original site. For links that leave the SCC, the browser repeats the process from the beginning with the embedded key. Everything needed to display and navigate the site is thus contained in the hashed values themselves. As with any Merkle structure, editing a page changes the keys of that page's SCC and of all pages from which it can be reached, and of no others.

8 Related Work

Merkle hashing of cyclic definitions. The core idea of the digraph Merkle hashing algorithm was independently discovered by the developers of the Unison programming language [2], who hash the abstract syntax trees of mutually recursive definitions. Each function and datatype is a node in a digraph, with its AST as the node label, and each cycle of mutually recursive definitions is hashed as a unit. To our knowledge their scheme does not treat non-trivial automorphism orbits inside a cycle, which arise, for example, from two structurally identical mutually recursive definitions that differ only in name. Such orbits appear to be rare in practice, but a complete treatment requires either the orbit index of Algorithm 3.2 or the local Merkle hash of Section 5.5, in which non-trivial orbits

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

never arise (Lemma 5.8). Hash-consing of cyclic structures and unique representations of regular trees [26, 14], canonical forms of recursive types [16], and hashing of terms modulo α-equivalence [27] address closely related problems in programming language implementation.

Color refinement and Weisfeiler–Leman. The Weisfeiler–Lehman graph hash [40] iteratively hashes and aggregates the neighborhoods of each node of an undirected graph. Isomorphic graphs are guaranteed to receive equal hashes, but the converse fails: 1-dimensional Weisfeiler–Leman refinement [43] cannot distinguish, for example, two disjoint triangles from a 6-cycle, or any two regular graphs of the same order and degree. In the kernel literature the number of iterations h is a parameter that controls the depth of the subtree features. When refinement is used to test or hash equivalence, as here, no parameter is needed, because refinement stops at the coarsest stable partition after at most $n - 1$ splits. Efficient implementations of color refinement run in $O((n + m) \log n)$ time [13, 8]. The local hash of Section 5 refines along outgoing edges with the canonical colour refinement of [8], and its hashes can be compared across graphs. The equivalence it computes is the view equivalence of anonymous networks [44] and the fibration equivalence of Boldi and Vigna [10], and for undirected graphs it corresponds to isomorphism of universal covers [32].

Bisimulation and automata. Bisimilarity is computed by relational coarsest partition algorithms [21, 35], and state equivalence of deterministic automata by Hopcroft-style minimization [17, 23, 42]. Sections 5–4 use these as the first stage of canonical hashing.

Canonization. Canonical labeling tools [28, 29, 20, 15] and Babai's quasipolynomial-time isomorphism test and canonization [6, 7] underlie Section 3. For ordered undirected graphs, in which the edges at each node are cyclically ordered, Jiang and Bunke [18, 19] give a canonical code computed in $O(m^2)$ time from every starting edge, and Booth [11] showed that isomorphism of finite automata is as hard as graph isomorphism (Section 6).

Arshad et al. In a 2014 paper, Arshad et al. [5] presented an algorithm for hashing directed graphs given additional inputs: a fixed source node and a depth-first search traversal tree. The traversal tree classifies the edges as tree, back, forward, and cross edges depending on search order, and the hash uses the traversal numbers to order the nodes. The authors note that isomorphic graphs may hash to different values because of this dependence on the source node and DFS tree. Their approach can be used to hash nodes with ordered outgoing edges and can be viewed as an alternative to Algorithm 6.1.

In a separate 2018 paper, Arshad et al. [4] presented a graph hashing algorithm in which the hash of a node combines, by XOR, the hashes of the labels of all of that node's descendants; the graph hash is then computed by hashing each node together with an accumulated graph hash. As described, this approach has two weaknesses. First, XOR aggregation is not a multiset hash: an even number of identically labeled descendants cancels out, so a node with two identically labeled descendants contributes as if it had none. Second, the aggregation ignores the edge structure among the descendants. For example, in the directed k-cycle and in the complete digraph on k vertices, with all labels equal, every node has the same $k - 1$ identically labeled descendants, so the algorithm outputs equal hashes for these non-isomorphic graphs.

9 Experimental Results We implemented the algorithms of this paper in the dihash Python library[3]. Global canonization and orbits use nauty through pynauty [28]. The local hashes use a simpler version of canonical colour refinement that splits blocks by sorting, which takes $O((n+m) \log^2 n)$ time, and, for bisimilarity, the algorithm of Paige and Tarjan. Ordered local hashing computes the minimal automaton with a canonical version of Hopcroft's algorithm and orders each component with it as well. All hashes use SHA-256 over an injective encoding. Running times were measured with Python 3.13 on a single core of an Intel Xeon processor at 2.1 GHz.

Validation. The test suite of the library compares every hash with an independent brute-force decision of the corresponding equivalence:

- isomorphism and rooted isomorphism, by enumerating all bijections;
- unfolding isomorphism, ordered or annotated with the internal/external marks of Section 5.5, by comparing canonical forms of truncated unfoldings of depth $2(n_1 + n_2) + 2$;
- bisimilarity, plain or annotated, by a greatest-fixpoint computation;
- rooted ordered isomorphism, by simultaneous traversal;

[3] https://github.com/calebh/dihash

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Table 9.1: Validation against brute-force equivalence checks: number of node pairs compared, with no disagreements in any row. Rows marked with * include multidigraphs. The component hashes were compared on every pair of strongly connected components.

Scheme (result)	Pairs checked
Global graph hash (Thm. 3.1)	3,000 graphs
Global node hash (Thm. 3.2)	40,206
Local hash* (Thm. 5.15)	80,497
Bisimulation hash* (Thm. 5.17)	80,497
Local component hashes, both* (Thms. 5.9, 5.17)	80,497
Ordered node hash* (Thm. 6.2)	79,011
Ordered local hash* (Thm. 6.3)	79,011
Ordered component hashes, global and local* (Thms. 6.4, 6.3)	79,011
Merkle hashes, global, local and bisimulation (Thm. 4.2, Prop. 5.11)	41,259
Ordered Merkle hashes, global and local* (Thm. 4.2, Prop. 5.11)	80,127

- global Merkle equivalence, by a direct implementation of Definition 4.1.

For Table 9.1 we ran these comparisons on 3000 random pairs of labeled graphs per scheme, each with 1–6 nodes, edge probability 0.15, 0.3 or 0.45, self-loops allowed, and one or two labels. In 40% of the pairs the second graph is a copy of the first with renamed nodes and a random insertion order. Where marked, half of the pairs are multidigraphs, in which some nodes have parallel edges, and ordered graphs then link to the same target more than once. Every pair of nodes of the two graphs was compared, and no disagreement was found. The test suite also confirms the separating examples of Proposition 5.12, the counterexample of Example 6.1, and the orbits of Figure 2.1. It checks that the canonical refinement is invariant under renumbering of the nodes, and that the local hashes do not change when a node is split into two copies that share its outgoing edges and divide its incoming edges, which preserves all unfoldings.

Scaling. Table 9.2 reports running times on three families of ordered digraphs. The random digraphs have two labels and $2n$ randomly drawn edges, with duplicates discarded and ports in order of insertion; they have a giant strongly connected component containing 63–65% of the nodes. The second family is the directed n-cycle in which one node has a different label, a worst case for refinement by rounds of hashing, which needs about $n/2$ rounds there. The third family is the worst case of Algorithm 6.2: all labels are equal, the first port follows a random n-cycle and the second a random permutation, so every port permutes the nodes and all nodes lie in a single class of $\equiv$. For the random digraphs and the permutation graphs we report the median over three graphs. The unordered hashes ignore the ports. From $n = 4000$ on, when n quadruples, the running time of every local hash, and of the local Merkle hashes, grows by a factor between 3.5 and 6.6, and graphs with 64,000 nodes are hashed in at most 5 s. Computing $\mathcal{H}_O$ for every node runs a depth-first search from every node, and its running time grows by a factor of 17 to 20 when n quadruples, i.e. quadratically. The global Merkle hash of an ordered graph runs Algorithm 6.2 on every component. On the random digraphs and on the cycle the classes C are small, and this hash scales like the local hashes. On the permutation graphs C contains all n nodes, and its running time grows quadratically, like that of $\mathcal{H}_O$ for every node.

Granularity. To illustrate how the equivalences compare, we hashed 400 random unlabeled digraphs with 12 nodes and 18 randomly drawn edges (duplicates discarded) and counted the distinct hash values per graph. On average the graphs have 12 nodes and the following numbers of classes:

- 11.85 orbits (global node hash);
- 9.94 $\approx_{\text{glob}}$ classes (global Merkle hash);
- 9.76 $\approx_{\text{loc}}$ classes (local Merkle hash);
- 9.19 unfolding classes (local hash);
- 7.78 bisimulation classes.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

Table 9.2: Time in seconds to hash all nodes of a graph with n nodes with the dihash library. The columns are the local hash $\mathcal{H}_L$, the bisimulation hash $\mathcal{H}_B$, the ordered local hash, the local Merkle hash of the graph as an unordered and as an ordered graph, $\mathcal{H}_O$ for every node, and the global Merkle hash of the ordered graph. The permutation graphs (perm.) are the worst case of Algorithm 6.2. A dash marks runs that were not attempted.

Graph	n	largest SCC	$\mathcal{H}_L$	$\mathcal{H}_B$	ordered local	$\mathcal{H}_M^{\text{loc}}$ (unord.)	$\mathcal{H}_M^{\text{loc}}$ (ord.)	$\mathcal{H}_O$, all nodes	$\mathcal{H}_M^{\text{glob}}$ (ord.)
random	1000	654	0.020	0.022	0.019	0.038	0.036	4.775	0.044
random	4000	2546	0.101	0.112	0.104	0.223	0.221	86.19	0.196
random	16000	10050	0.539	0.518	0.525	0.779	0.875	–	1.011
random	64000	40287	2.826	3.435	3.025	4.189	4.255	–	4.637
cycle	1000	1000	0.019	0.016	0.042	0.054	0.052	5.877	0.029
cycle	4000	4000	0.069	0.067	0.069	0.077	0.084	100.67	0.096
cycle	16000	16000	0.324	0.318	0.293	0.388	0.353	–	0.398
cycle	64000	64000	1.812	1.897	1.561	1.548	1.515	–	1.843
perm.	1000	1000	0.004	0.003	0.004	0.017	0.018	7.574	7.842
perm.	4000	4000	0.013	0.015	0.017	0.091	0.096	153.12	155.86
perm.	16000	16000	0.064	0.077	0.102	0.393	0.425	–	–
perm.	64000	64000	0.368	0.473	0.522	1.872	2.188	–	–

This is consistent with the implications rooted isomorphism $\Rightarrow \approx_{\text{glob}} \Rightarrow \approx_{\text{loc}} \Rightarrow$ unfolding isomorphism $\Rightarrow$ bisimilarity (Proposition 5.12 and Section 5). The gaps show that the choice of equivalence matters in practice even on small random graphs.

10 Conclusion We have given hashing schemes for node-labeled digraphs whose hash equality corresponds exactly, up to collisions of the underlying string hash, to isomorphism, rooted isomorphism, unfolding isomorphism, bisimilarity, and two Merkle-style equivalences, together with fast variants for graphs with ordered outgoing edges. The local hashes are computed in $O((n+m)\log n)$ time by applying the local Merkle hash to the unfolding quotient, and the local Merkle variant combines the incrementality of Merkle hashing with the same running time. Natural directions for future work include incremental computation of local hashes, a canonization of strongly connected ordered graphs in subquadratic time also in the worst case, and evaluation on real data such as computer program call graphs and web crawls.

Acknowledgements Early versions of this paper were posted on arXiv in 2020, and some preliminary results were presented at the 51st Southeastern International Conference on Combinatorics, Graph Theory, and Computing (SEICCGTC). These results were not published in the proceedings and were not peer reviewed. If you are looking for an earlier version of the paper, see the arXiv page at arXiv:2002.06653.

Per SIAM's policy on AI disclosure, we disclose that we utilized Claude Opus 5.5 to transform proof sketches to full proofs in Section 5 and Section 4. Opus 5.5 was utilized to simplify a human written proof to the one seen in Theorem 3.2. Opus 5.5 was used to tidy up other sections of the paper and as simulated peer review. The core algorithms presented and the code implementations have been under development by the author of this paper since 2015, and this paper is a product of many years of human effort.

References

[1] *IPFS is the distributed web*, https://ipfs.io/. Accessed: 2019-12-09.

[2] *Unison programming language*, https://www.unisonweb.org/. Accessed: 2019-12-09.

[3] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley, 1974.

[4] M. U. Arshad, A. Kundu, E. Bertino, A. Ghafoor, and C. Kundu, *Efficient and scalable integrity verification of data and query results for graph databases*, IEEE Trans. Knowl. Data Eng., 30 (2018), pp. 866–879.

[5] M. U. Arshad, A. Kundu, E. Bertino, K. Madhavan, and A. Ghafoor, *Security of graph data: hashing schemes and definitions*, in Proceedings of the 4th ACM conference on Data and application security

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

and privacy, CODASPY '14, New York, NY, USA, Mar. 2014, Association for Computing Machinery, pp. 223–234.

[6] L. Babai, *Graph isomorphism in quasipolynomial time [extended abstract]*, in Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, STOC '16, New York, NY, USA, 2016, Association for Computing Machinery, p. 684–697, https://doi.org/10.1145/2897518.2897542, https://doi.org/10.1145/2897518.2897542.

[7] L. Babai, *Canonical form for graphs in quasipolynomial time: preliminary report*, in Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, New York, NY, USA, 2019, Association for Computing Machinery, p. 1237–1246, https://doi.org/10.1145/3313276.3316356, https://doi.org/10.1145/3313276.3316356.

[8] C. Berkholz, P. Bonsma, and M. Grohe, *Tight lower and upper bounds for the complexity of canonical colour refinement*, Theory of Computing Systems, 60 (2017), pp. 581–614.

[9] N. Bloyet, P.-F. Marteau, and E. Frénod, *Scott: A method for representing graphs as rooted trees for graph canonization*, in Complex Networks and Their Applications VIII, Studies in computational intelligence, Springer International Publishing, Cham, 2020, pp. 578–590.

[10] P. Boldi and S. Vigna, *Fibrations of graphs*, Discrete Math., 243 (2002), p. 21–66, https://doi.org/10.1016/S0012-365X(00)00455-6, https://doi.org/10.1016/S0012-365X(00)00455-6.

[11] K. S. Booth, *Isomorphism testing for graphs, semigroups, and finite automata are polynomially equivalent problems*, SIAM Journal on Computing, 7 (1978), pp. 273–279, https://doi.org/10.1137/0207023.

[12] K. S. Booth, *Lexicographically least circular substrings*, Information Processing Letters, 10 (1980), pp. 240–242, https://doi.org/10.1016/0020-0190(80)90149-0.

[13] A. Cardon and M. Crochemore, *Partitioning a graph in $O(|A| \log_2 |V|)$*, Theoretical Computer Science, 19 (1982), pp. 85–98.

[14] J. Considine, *Efficient hash-consing of recursive types*, tech. report, Boston University, 2000.

[15] P. T. Darga, M. H. Liffiton, K. A. Sakallah, and I. L. Markov, *Exploiting structure in symmetry detection for CNF*, in Proceedings of the 41st annual Design Automation Conference, ACM, 2004, pp. 530–534.

[16] N. Gauthier and F. Pottier, *Numbering matters: First-order canonical forms for second-order recursive types*, in Proceedings of the Ninth ACM SIGPLAN International Conference on Functional Programming (ICFP), 2004, https://doi.org/10.1145/1016850.1016872.

[17] J. Hopcroft, *An n log n algorithm for minimizing states in a finite automaton*, in Theory of Machines and Computations, Z. Kohavi and A. Paz, eds., Academic Press, 1971, pp. 189–196, https://doi.org/10.1016/B978-0-12-417750-5.50022-1, https://www.sciencedirect.com/science/article/pii/B9780124177505500221.

[18] X. Jiang and H. Bunke, *On the coding of ordered graphs*, Computing, 61 (1998), pp. 23–38, https://doi.org/10.1007/BF02684448.

[19] X. Jiang and H. Bunke, *Optimal quadratic-time isomorphism of ordered graphs*, Pattern Recognition, 32 (1999), pp. 1273–1283, https://doi.org/10.1016/S0031-3203(98)00145-9.

[20] T. Junttila and P. Kaski, *Engineering an efficient canonical labeling tool for large and sparse graphs*, in 2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX), SIAM, 2007, pp. 135–149.

[21] P. C. Kanellakis and S. A. Smolka, *Ccs expressions, finite state processes, and three problems of equivalence*, (1983), p. 228–240, https://doi.org/10.1145/800221.806724, https://doi.org/10.1145/800221.806724.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

[22] J.-Y. Kao, N. Rampersad, and J. Shallit, *On nfas where all states are final, initial, or both*, Theor. Comput. Sci., 410 (2009), p. 5010–5021, https://doi.org/10.1016/j.tcs.2009.07.049, https://doi.org/10.1016/j.tcs.2009.07.049.

[23] T. Knuutila, *Re-describing an algorithm by hopcroft*, Theor. Comput. Sci., 250 (2001), pp. 333–363.

[24] D. Kőnig, *Theorie der endlichen und unendlichen Graphen*, Akademische Verlagsgesellschaft, Leipzig, 1936.

[25] J. Loeliger and M. McCullough, *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*, "O'Reilly Media, Inc.", Aug. 2012.

[26] L. Mauborgne, *An incremental unique representation for regular trees*, Nordic J. of Computing, 7 (2000), p. 290–311.

[27] K. Maziarz, T. Ellis, A. Lawrence, A. Fitzgibbon, and S. Peyton Jones, *Hashing modulo alpha-equivalence*, in Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2021, New York, NY, USA, 2021, Association for Computing Machinery, p. 960–973, https://doi.org/10.1145/3453483.3454088, https://doi.org/10.1145/3453483.3454088.

[28] B. D. McKay, *Practical graph isomorphism*, Department of Computer Science, Vanderbilt University Tennessee, USA, 1981.

[29] B. D. Mckay and A. Piperno, *Practical graph isomorphism, ii*, J. Symb. Comput., 60 (2014), p. 94–112, https://doi.org/10.1016/j.jsc.2013.09.003, https://doi.org/10.1016/j.jsc.2013.09.003.

[30] R. C. Merkle, *A digital signature based on a conventional encryption function*, in Conference on the theory and application of cryptographic techniques, Springer, 1987, pp. 369–378.

[31] S. Nakamoto, *Bitcoin: A peer-to-peer electronic cash system*, (2008).

[32] N. Norris, *Universal covers of graphs: Isomorphism to depth $n-1$ implies isomorphism to all depths*, Discrete Applied Mathematics, 56 (1995), pp. 61–74.

[33] N. I. of Standards and Technology, *Secure hash standard*, Tech. Report Publication 180-2, 100 Bureau Drive, Stop 8900, Gaithersburg, MD 20899-8900, August 2002, https://csrc.nist.gov/csrc/media/publications/fips/180/2/archive/2002-08-01/documents/fips180-2.pdf.

[34] N. I. of Standards and Technology, *SHA-3 standard: Permutation-based hash and extendable-output functions*, Tech. Report Publication 202, 100 Bureau Drive, Stop 8900, Gaithersburg, MD 20899-8900, August 2015, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

[35] R. Paige and R. E. Tarjan, *Three partition refinement algorithms*, SIAM Journal on Computing, 16 (1987), pp. 973–989, https://doi.org/10.1137/0216062, https://doi.org/10.1137/0216062, https://arxiv.org/abs/https://doi.org/10.1137/0216062.

[36] C. Picard and R. Matthes, *Coinductive graph representation: the problem of embedded lists*, Electronic Communications of the EASST, 39 (2011), https://doi.org/10.14279/tuj.eceasst.39.649, https://eceasst.org/index.php/eceasst/article/view/2664.

[37] R. Rivest, *The MD5 message-digest algorithm*, Tech. Report RFC 1321, MIT Laboratory for Computer Science and RSA Data Security, Inc., 545 Technology Square, Cambridge, MA 02139-1986, April 1992, https://tools.ietf.org/html/rfc1321.

[38] D. Sangiorgi, *Introduction to Bisimulation and Coinduction*, Cambridge University Press, 2011.

[39] N. Shervashidze and K. Borgwardt, *Fast subtree kernels on graphs*, in Advances in Neural Information Processing Systems, Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta, eds., vol. 22, Curran Associates, Inc., 2009, https://proceedings.neurips.cc/paper_files/paper/2009/file/0a49e3c3a03ebde64f85c0bacd8a08e2-Paper.pdf.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited

[40] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, *Weisfeiler-Lehman graph kernels*, J. Mach. Learn. Res., 12 (2011).

[41] R. Tarjan, *Depth-first search and linear graph algorithms*, SIAM Journal on Computing, 1 (1972), pp. 146–160, https://doi.org/10.1137/0201010, https://doi.org/10.1137/0201010, https://arxiv.org/abs/https://doi.org/10.1137/0201010.

[42] A. Valmari and P. Lehtinen, *Efficient minimization of DFAs with partial transition functions*, in Proceedings of the 25th International Symposium on Theoretical Aspects of Computer Science (STACS), vol. 1 of LIPIcs, 2008, pp. 645–656.

[43] B. Weisfeiler and A. Leman, *The reduction of a graph to canonical form and the algebra which appears therein*, Nauchno-Technicheskaya Informatsia, Series 2, 9 (1968), pp. 12–16. English translation available.

[44] M. Yamashita and T. Kameda, *Computing on anonymous networks. i. characterizing the solvable cases*, IEEE Transactions on Parallel and Distributed Systems, 7 (1996), pp. 69–89, https://doi.org/10.1109/71.481599.

Copyright © 20XX by SIAM
Unauthorized reproduction of this article is prohibited